

1) Setup Jenkins CICD pipeline using freestyle job using Docker containers using below code.

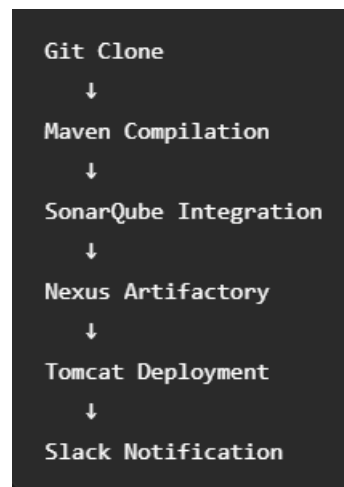
Source Code: <https://github.com/betawins/hiring-app.git>

Stages:

- Git Clone
- SonarQube Integration
- Maven Compilation
- Nexus Artifactory
- Slack Notification
- Deploy On tomcat

1. Objective

To set up a Jenkins Freestyle CI/CD pipeline using Docker containers and automate the following stages:



Source Code

<https://github.com/betawins/hiring-app.git>

Jenkins Job

Hiring-App-Pipeline

2. Environment Setup

The CI/CD environment was created on an **Amazon Linux 2023 EC2 instance**.

Environment

```
OS           : Amazon Linux 2023
Jenkins      : Port 8080
SonarQube    : Port 9000
Nexus        : Port 8081
Tomcat       : Port 8082
Java         : Amazon Corretto 21
Docker       : Used for Maven, SonarQube, Nexus and Tomcat
```

3. Connect to EC2

Connected to the Amazon Linux EC2 instance using SSH.

4. Verify Linux Version

```
cat /etc/os-release
```

Verified that the server was running **Amazon Linux 2023**.

5. Install Java 21

Installed Amazon Corretto 21:

```
sudo dnf install java-21-amazon-corretto -y
```

Verified Java:

```
java -version
```

Configured:

```
export JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto.x86_64
```

Verified:

```
echo $JAVA_HOME
```

Observation

Java 21 was successfully installed and available for Jenkins.

6. Install Jenkins

Downloaded the Jenkins repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \  
https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

Imported the Jenkins key:

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2026.key
```

Installed Jenkins:

```
sudo dnf install jenkins -y
```

Enabled and started Jenkins:

```
sudo systemctl enable jenkins  
sudo systemctl start jenkins
```

Checked Jenkins:

```
sudo systemctl status jenkins
```

Jenkins was accessed through:

<http://<EC2-Public-IP>:8080>

7. Configure Jenkins Port

Jenkins was configured to use:

8080

Restarted Jenkins:

```
sudo systemctl restart jenkins
```

Verified:

```
sudo systemctl status jenkins
```

8. Install Docker

Installed Docker:

```
sudo dnf install docker -y
```

Enabled and started Docker:

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

Verified:

```
docker --version
```

Checked Docker service:

```
sudo systemctl status docker
```

9. Allow Jenkins to Use Docker

Added Jenkins to the Docker group:

```
sudo usermod -aG docker jenkins
```

Restarted Jenkins:

```
sudo systemctl restart jenkins
```

Verified Docker access as Jenkins:

```
sudo -u jenkins docker ps
```

Observation

Jenkins successfully gained permission to execute Docker commands.

10. Install Git

Installed Git:

```
sudo dnf install git -y
```

Verified:

```
git --version
```

Git was required for Jenkins to clone the GitHub repository.

11. Run SonarQube Using Docker

Pulled the SonarQube image:

```
docker pull sonarqube:lts-community
```

Created the SonarQube container:

```
docker run -d \  
  --name sonarqube \  
  -p 9000:9000 \  
  sonarqube:lts-community
```

Verified:

```
docker ps
```

SonarQube was accessed through:

<http://<EC2-Public-IP>:9000>

12. Run Nexus Using Docker

Pulled Nexus:

```
docker pull sonatype/nexus3
```

Created the Nexus container:

```
docker run -d \  
  --name nexus \  
  -p 8081:8081 \  
  sonatype/nexus3
```

Verified:

```
docker ps
```

Nexus was accessed through:

<http://<EC2-Public-IP>:8081>

13. Get Nexus Initial Password

Retrieved the initial administrator password:

```
docker exec nexus cat /nexus-data/admin.password
```

Used it to log in to Nexus.

14. Create Nexus Maven Repository

In Nexus:

Repositories → Create repository → maven2 (hosted)

Configured:

Repository name : maven-release

Version policy : Release

Deployment : Disable redeploy

Repository URL:

<http://<EC2-Public-IP>:8081/repository/maven-release/>

15. Run Tomcat Using Docker

Pulled Tomcat 10.1:

```
docker pull tomcat:10.1
```

Created the Tomcat container:

```
docker run -d \  
  --name tomcat \  
  -p 8082:8080 \  
  tomcat:10.1
```

Verified:

```
docker ps
```

Tomcat was accessed through:

<http://<EC2-Public-IP>:8082>

16. Configure Tomcat Manager

Copied the Manager application:

```
docker exec tomcat cp -r \  
/usr/local/tomcat/webapps.dist/manager \  
/usr/local/tomcat/webapps/
```

Configured the Tomcat Manager user with the required:

```
manager-script
```

role.

Created the Jenkins deployment user in Tomcat.

Tomcat Manager was verified through:

<http://<EC2-Public-IP>:8082/manager/html>

17. Install Jenkins Plugins

Installed the required Jenkins plugins:

Git plugin

Used for GitHub source-code checkout.

SonarQube Scanner for Jenkins

Used to integrate Jenkins with SonarQube.

Slack Notification plugin

Used to send Jenkins build notifications to Slack.

Deploy to Container plugin

Checked for Tomcat deployment, but the available versions did not support Tomcat 10.1, so Tomcat Manager API was used instead.

STAGE 1 — GIT CLONE

18. Create Freestyle Jenkins Job

Created:

Hiring-App-Pipeline

Selected:

Freestyle project

19. Configure Git Clone

Under:

Source Code Management → Git

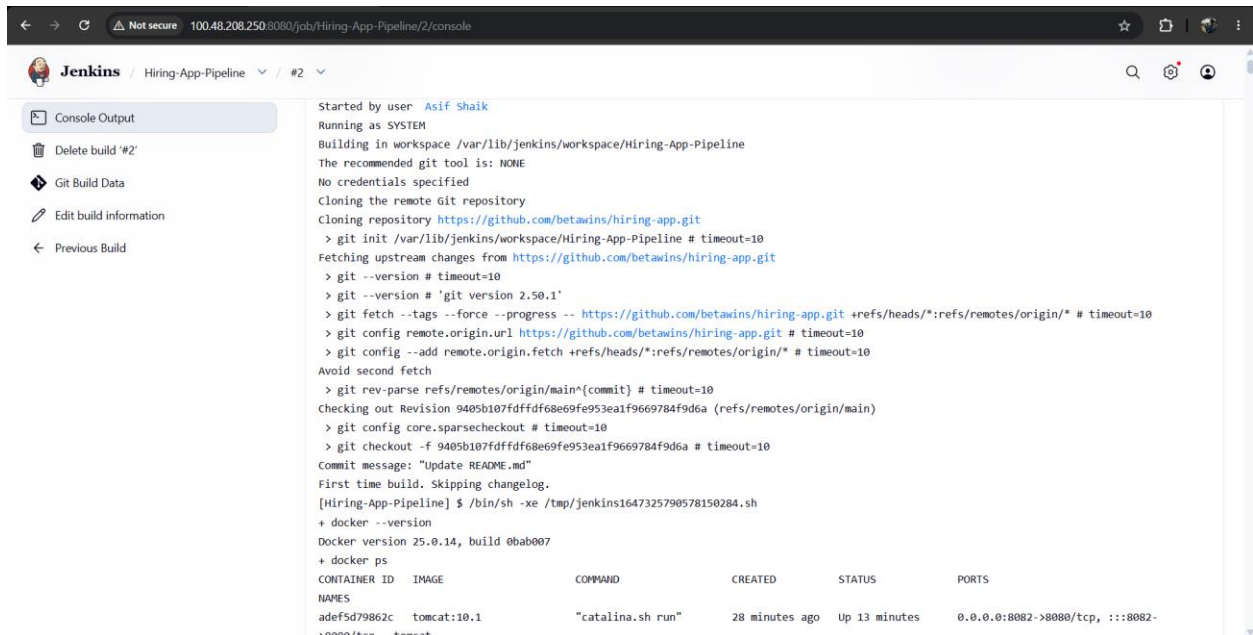
Repository:

<https://github.com/betawins/hiring-app.git>

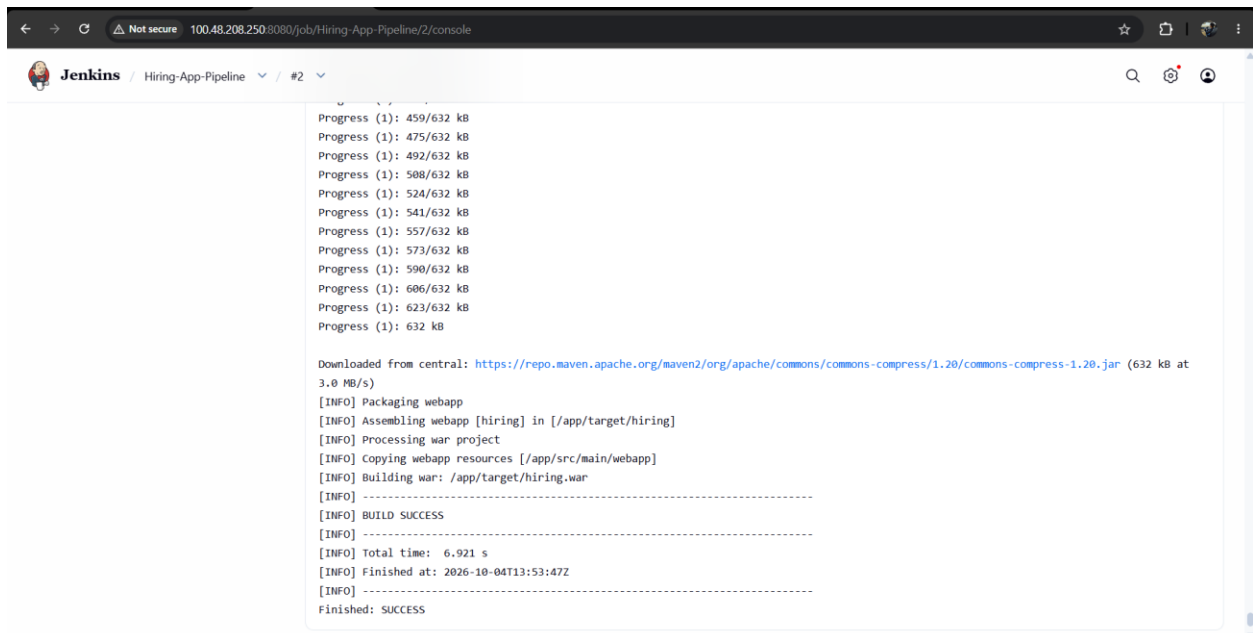
Branch:

*/main

Jenkins cloned the application into the workspace.

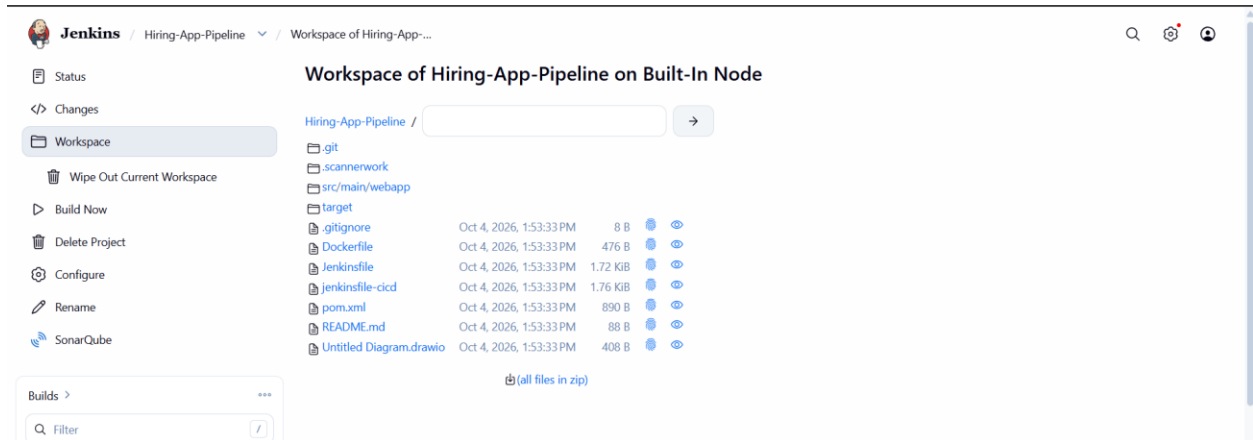


```
Started by user Asif Shaik
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Hiring-App-Pipeline
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/hiring-app.git
> git init /var/lib/jenkins/workspace/Hiring-App-Pipeline # timeout=10
Fetching upstream changes from https://github.com/betawins/hiring-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/hiring-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953eaf9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953eaf9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[Hiring-App-Pipeline] $ /bin/sh -xe /tmp/jenkins1647325790578150284.sh
+ docker --version
Docker version 25.0.14, build 0bab007
+ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAME
adeF5d79862c   tomcat:10.1    "catalina.sh run"        28 minutes ago Up 13 minutes 0.0.0.0:8082->8080/tcp, :::8082->8080/tcp
>8080/trn      tomcat
```



```
Progress (1): 459/632 kB
Progress (1): 475/632 kB
Progress (1): 492/632 kB
Progress (1): 508/632 kB
Progress (1): 524/632 kB
Progress (1): 541/632 kB
Progress (1): 557/632 kB
Progress (1): 573/632 kB
Progress (1): 590/632 kB
Progress (1): 606/632 kB
Progress (1): 623/632 kB
Progress (1): 632 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.20/commons-compress-1.20.jar (632 kB at 3.0 MB/s)
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/app/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/app/src/main/webapp]
[INFO] Building war: /app/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.921 s
[INFO] Finished at: 2026-10-04T13:53:47Z
[INFO] -----
Finished: SUCCESS
```



Flow

GitHub



Jenkins



Workspace

Observation

The application source code was successfully cloned from GitHub into the Jenkins workspace.

Result

Stage 1 — Git Clone completed successfully.

STAGE 2 — MAVEN COMPILATION

20. Maven Compilation Using Docker

Added:

Build Steps → Execute shell

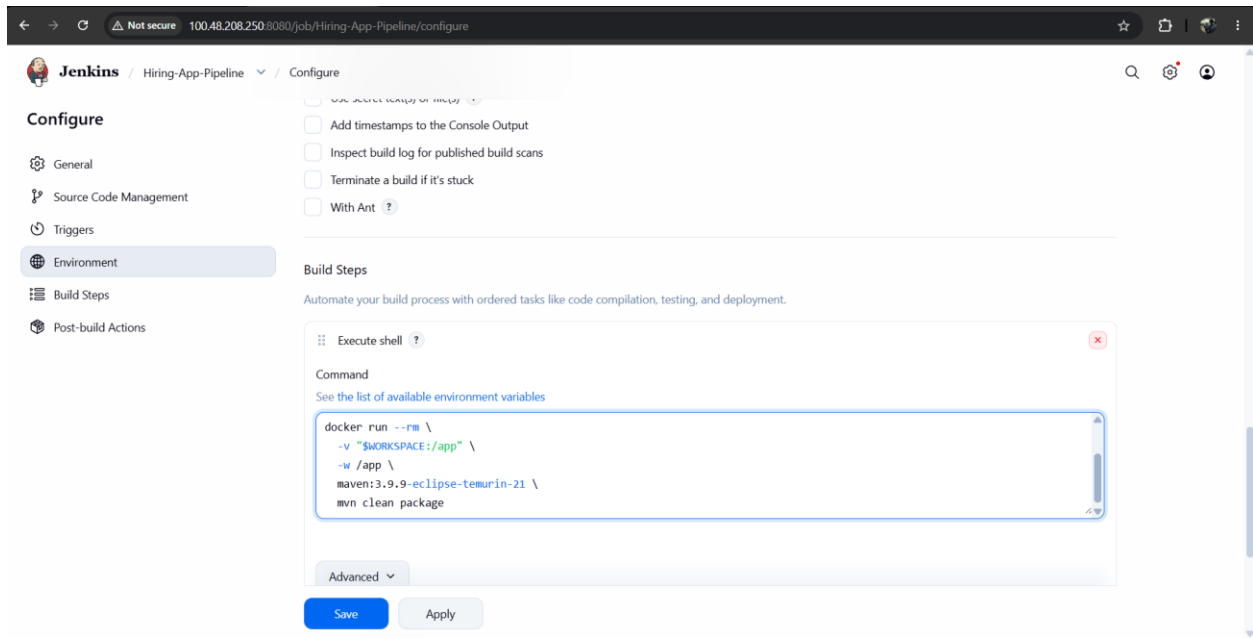
docker --version

docker ps

```
docker run --rm \
-v "$WORKSPACE:/app" \
-w /app \
maven:3.9.9-eclipse-temurin-21 \
mvn clean package
```

The Maven build generated:

Target/hiring.war



The screenshot shows the Jenkins web interface for configuring a job named 'Hiring-App-Pipeline'. The left sidebar contains navigation links: 'General', 'Source Code Management', 'Triggers', 'Environment' (which is selected), 'Build Steps', and 'Post-build Actions'. The main area is titled 'Configure' and includes several checkboxes: 'Add timestamps to the Console Output', 'Inspect build log for published build scans', 'Terminate a build if it's stuck', and 'With Ant'. Below these is the 'Build Steps' section, which contains a single step named 'Execute shell'. The 'Command' field for this step contains the Docker command: `docker run --rm \`
`-v "$WORKSPACE:/app" \`
`-w /app \`
`maven:3.9.9-eclipse-temurin-21 \`
`mvn clean package`. At the bottom of the configuration area are 'Save' and 'Apply' buttons.



The screenshot shows the Jenkins console output for the 'Hiring-App-Pipeline' job. The output displays a series of log messages with timestamps, indicating the successful execution of the build step. The messages include: `14:22:56.791 INFO Sensor VB.NET Properties [vbnet]`, `14:22:56.792 INFO Sensor VB.NET Properties [vbnet] (done) | time=1ms`, `14:22:56.792 INFO Sensor IaC Docker Sensor [iac]`, `14:22:56.794 INFO 0 source files to be analyzed`, `14:22:56.841 INFO 0/0 source files have been analyzed`, `14:22:56.841 INFO Sensor IaC Docker Sensor [iac] (done) | time=49ms`, `14:22:56.845 INFO ----- Run sensors on project`, `14:22:56.903 INFO Sensor Analysis Warnings import [csharp]`, `14:22:56.904 INFO Sensor Analysis Warnings import [csharp] (done) | time=1ms`, `14:22:56.905 INFO Sensor Zero Coverage Sensor`, `14:22:56.906 INFO Sensor Zero Coverage Sensor (done) | time=1ms`, `14:22:56.909 INFO SCM Publisher SCM provider for this project is: git`, `14:22:56.910 INFO SCM Publisher 2 source files to be analyzed`, `14:22:57.069 INFO SCM Publisher 2/2 source files have been analyzed (done) | time=159ms`, `14:22:57.072 INFO CPD Executor 1 file had no CPD blocks`, `14:22:57.072 INFO CPD Executor Calculating CPD for 0 files`, `14:22:57.073 INFO CPD Executor CPD calculation finished (done) | time=0ms`, `14:22:57.146 INFO Analysis report generated in 71ms, dir size=123.2 kB`, `14:22:57.162 INFO Analysis report compressed in 15ms, zip size=17.1 kB`, `14:22:57.709 INFO Analysis report uploaded in 546ms`, `14:22:57.710 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://100.48.208.250:9000/dashboard?id=hiring-app`, `14:22:57.710 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report`, `14:22:57.710 INFO More about the report processing at http://100.48.208.250:9000/api/ce/task?id=AaEHS-1VOiM9sX0Wmkk_`, `14:22:57.718 INFO Analysis total time: 7.036 s`, `14:22:57.719 INFO EXECUTION SUCCESS`, `14:22:57.720 INFO Total time: 10.498s`, and `Finished: SUCCESS`.

Flow

Jenkins Workspace



Docker Maven Container



mvn clean package



target/hiring.war

Observation

The application was successfully compiled and packaged using Maven inside a Docker container.

Result

Stage 2 — Maven Compilation completed successfully.

STAGE 3 — SONARQUBE INTEGRATION

21. Configure SonarQube

Created Jenkins credential:

sonarqube-token

Configured SonarQube in:

Manage Jenkins → System

Name:

sonarqube

URL:

<http://<EC2-Public-IP>:9000>

Configured SonarQube Scanner under:

Manage Jenkins → Tools

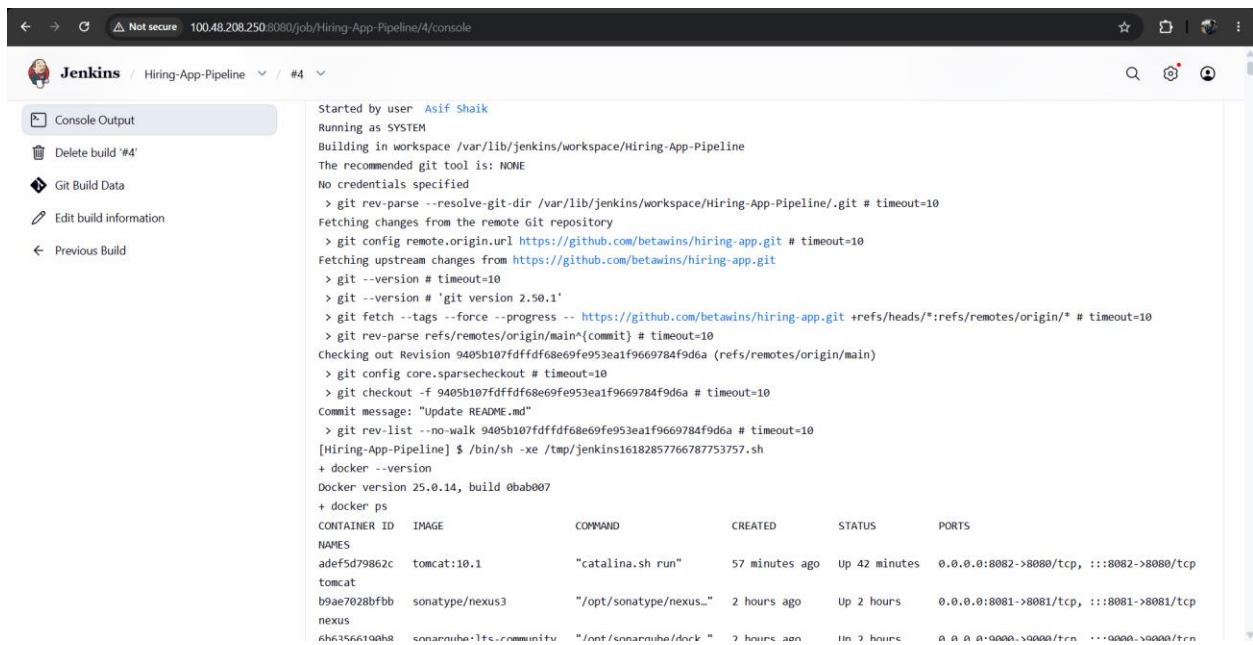
Name:
sonarqube-scanner

Added:

Build Steps → Execute SonarQube Scanner

Analysis properties:

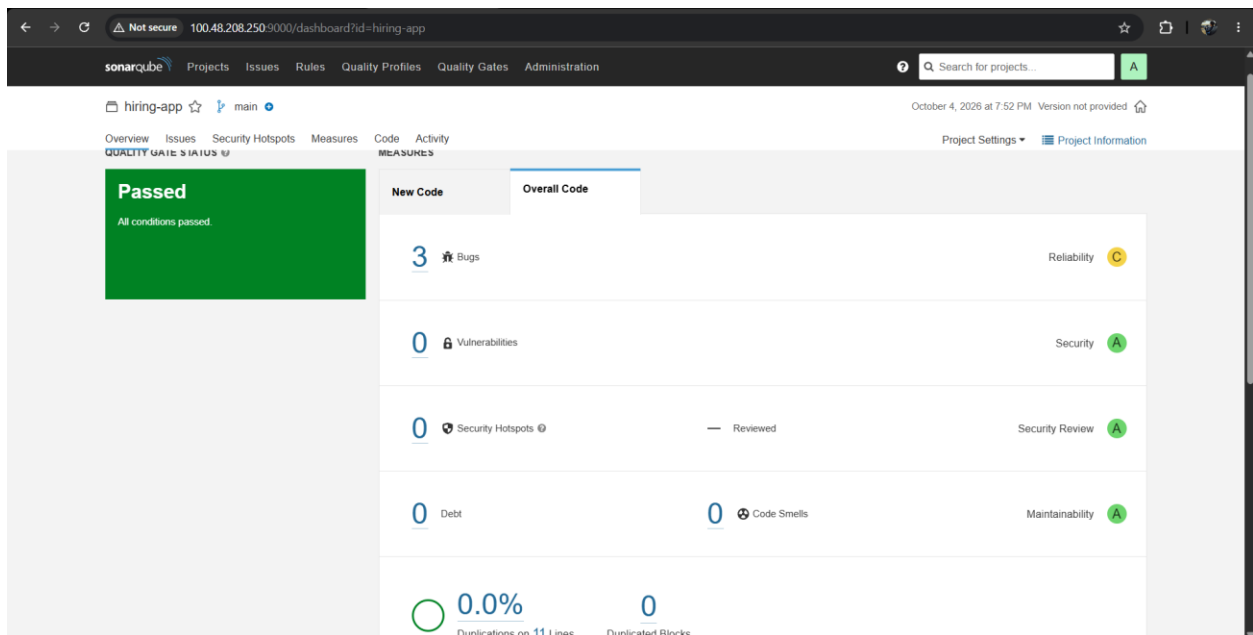
sonar.projectKey=hiring-app
sonar.projectName=hiring-app
sonar.sources=src/main



The screenshot shows the Jenkins web interface for a build step named 'Hiring-App-Pipeline' (step #4). The console output displays the execution of a Docker container and the SonarQube Scanner. The output includes the following text:

```
Started by user Asif Shaik
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Hiring-App-Pipeline
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Hiring-App-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
Fetching upstream changes from https://github.com/betawins/hiring-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/hiring-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953ea1f9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
[Hiring-App-Pipeline] $ /bin/sh -xe /tmp/jenkins16182857766787753757.sh
+ docker --version
Docker version 25.0.14, build 0bab007
+ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAME          adef5d79862c  tomcat:10.1            "catalina.sh run"   57 minutes ago Up 42 minutes 0.0.0.0:8082->8080/tcp, :::8082->8080/tcp
tomcat        b9ae7028bfbb  sonatype/nexus3        "/opt/sonatype/nexus-" 2 hours ago   Up 2 hours    0.0.0.0:8081->8081/tcp, :::8081->8081/tcp
nexus         4b1c56a19a8e  sonarqube-lts-community "/opt/sonarqube/dock-" 2 hours ago   Up 2 hours    0.0.0.0:9000->9000/tcp, :::9000->9000/tcp
```

```
14:22:56.791 INFO Sensor VB.NET Analysis Log [vbnet] (done) | time=11ms
14:22:56.791 INFO Sensor VB.NET Properties [vbnet]
14:22:56.792 INFO Sensor VB.NET Properties [vbnet] (done) | time=1ms
14:22:56.792 INFO Sensor Iac Docker Sensor [iac]
14:22:56.794 INFO 0 source files to be analyzed
14:22:56.841 INFO 0/0 source files have been analyzed
14:22:56.841 INFO Sensor Iac Docker Sensor [iac] (done) | time=49ms
14:22:56.845 INFO ----- Run sensors on project
14:22:56.903 INFO Sensor Analysis Warnings import [csharp]
14:22:56.904 INFO Sensor Analysis Warnings import [csharp] (done) | time=1ms
14:22:56.905 INFO Sensor Zero Coverage Sensor
14:22:56.906 INFO Sensor Zero Coverage Sensor (done) | time=1ms
14:22:56.909 INFO SCM Publisher SCM provider for this project is: git
14:22:56.910 INFO SCM Publisher 2 source files to be analyzed
14:22:57.069 INFO SCM Publisher 2/2 source files have been analyzed (done) | time=159ms
14:22:57.072 INFO CPD Executor 1 file had no CPD blocks
14:22:57.072 INFO CPD Executor Calculating CPD for 0 files
14:22:57.073 INFO CPD Executor CPD calculation finished (done) | time=0ms
14:22:57.146 INFO Analysis report generated in 71ms, dir size=123.2 kB
14:22:57.162 INFO Analysis report compressed in 15ms, zip size=17.1 kB
14:22:57.709 INFO Analysis report uploaded in 546ms
14:22:57.710 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://100.48.208.250:9000/dashboard?id=hiring-app
14:22:57.710 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
14:22:57.710 INFO More about the report processing at http://100.48.208.250:9000/api/ce/task?id=AaEH5-1VOiM9sXWmkk\_
14:22:57.718 INFO Analysis total time: 7.036 s
14:22:57.719 INFO EXECUTION SUCCESS
14:22:57.720 INFO Total time: 10.498s
Finished: SUCCESS
```



Observation

The hiring-app project was successfully analyzed and displayed in SonarQube.

Result

Stage 3 — SonarQube Integration completed successfully.

STAGE 4 — NEXUS ARTIFACTORY

22. Configure Nexus Credentials

Created Jenkins credential:

nexus-credentials

Configured username/password binding:

NEXUS_USERNAME

NEXUS_PASSWORD

These variables were used by the Nexus upload command.

23. Upload Artifact to Nexus

Created temporary Maven settings:

```
cat > "$WORKSPACE/settings.xml" <<EOF
<settings>
  <servers>
    <server>
      <id>nexus</id>
      <username>${NEXUS_USERNAME}</username>
      <password>${NEXUS_PASSWORD}</password>
    </server>
  </servers>
</settings>
EOF
```

Uploaded the WAR using Docker Maven:

```
docker run --rm \
  -v "$WORKSPACE:/app" \
  -w /app \
  maven:3.9.9-eclipse-temurin-21 \
  mvn deploy:deploy-file \
  --settings /app/settings.xml \
  -DgroupId=com.betawins \
  -DartifactId=hiring \
```

```
-Dversion=1.0.2 \  
-Dpackaging=war \  
-Dfile=target/hiring.war \  
-DrepositoryId=nexus \  
-Durl=http://<EC2-Public-IP>:8081/repository/maven-release/
```

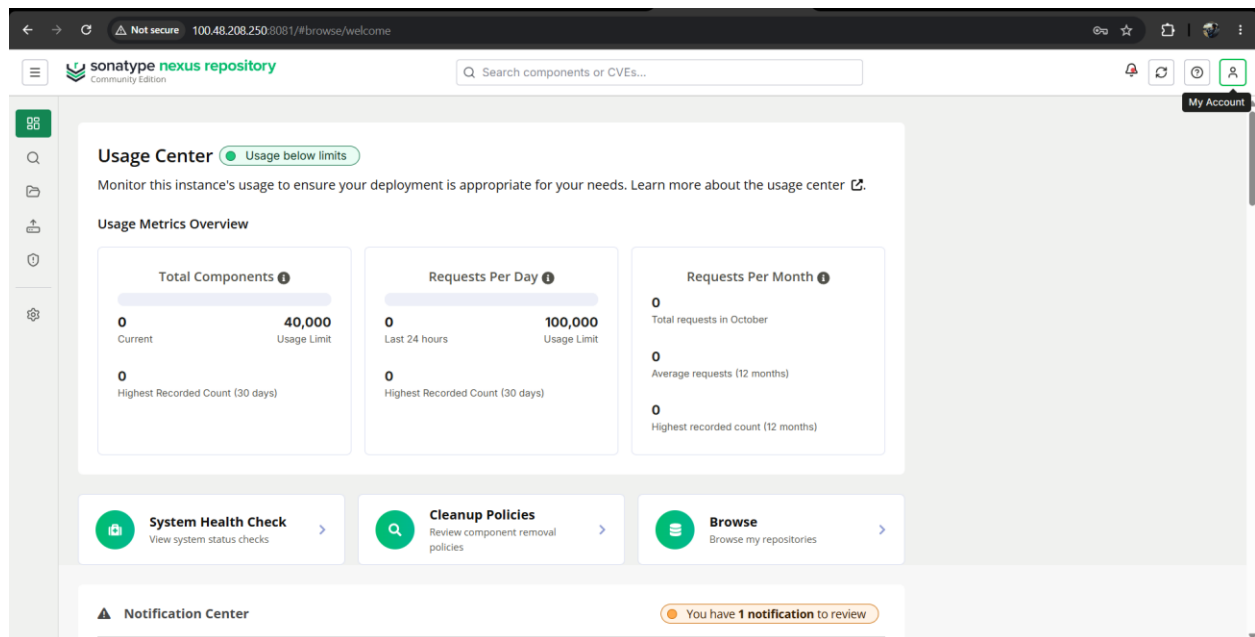
Removed temporary settings:

```
rm -f "$WORKSPACE/settings.xml"
```

Artifact

com.betawins:hiring:1.0.2

The WAR was successfully uploaded to Nexus.



sonatype nexus repository
Community Edition

Search components or CVEs...

Settings

- Repository
- Repositories
- Blob Stores
- Data Store
- Proprietary Repositories
- Content Selectors
- Cleanup Policies
- Routing Rules
- Security
- Support
- Logging
- Logs
- Recovery Mode
- Status
- Support ZIP
- System Information

Repositories / Select Recipe / Create Repository: maven2 (hosted)

Name: A unique identifier for this repository
maven-releases

Online: ☒ If checked, the repository accepts incoming requests

Maven 2

Version policy:
What type of artifacts does this repository store?
Release

Layout policy:
Validate that all paths are maven artifact or metadata paths
Strict

Content Disposition:
Add Content-Disposition header as 'Attachment' to disable some content from being inline in a browser.
Attachment

Storage

Blob store:
Blob store used to store repository contents
default

Strict Content Type Validation:
☒ Validate that all content uploaded to this repository is of a MIME type appropriate for the repository format

Jenkins / Hiring-App-Pipeline / #8

Status

Changes

Console Output

Delete build '#8'

Git Build Data

Edit build information

Previous Build

Console

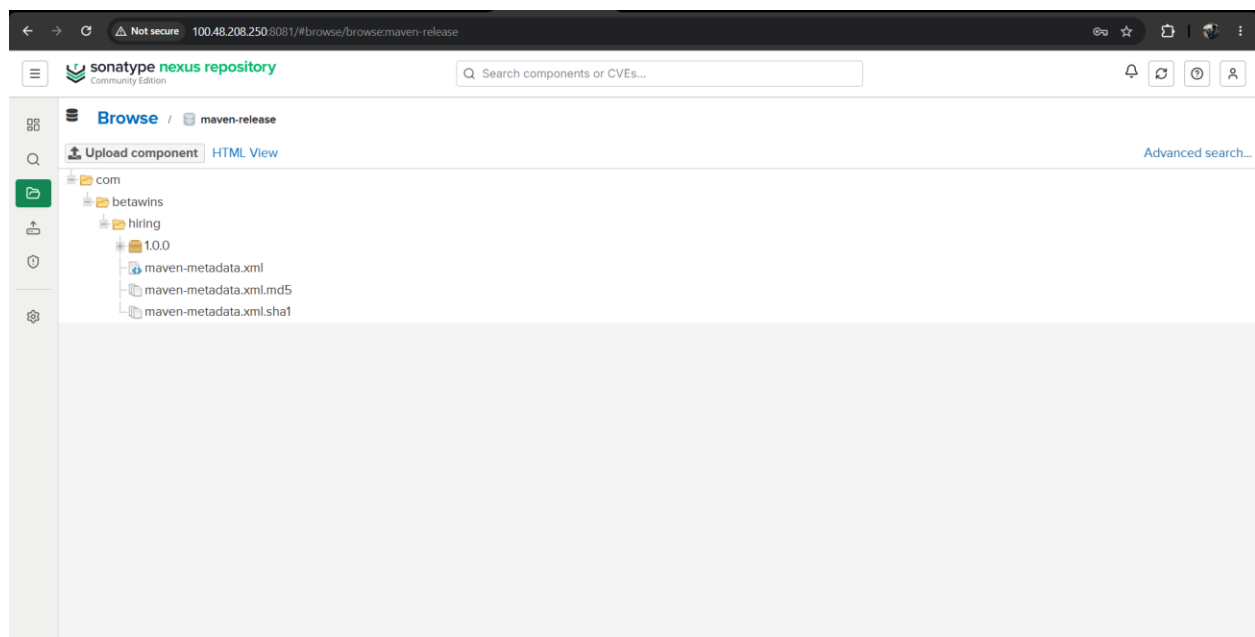
Download Copy View as plain text

This log is too long to show here, 14 KB has been skipped — click to see the complete log

```
Progress (1): 3.2/121 kB
Progress (1): 20/121 kB
Progress (1): 36/121 kB
Progress (1): 52/121 kB
Progress (1): 69/121 kB
Progress (1): 85/121 kB
Progress (1): 101/121 kB
Progress (1): 118/121 kB
Progress (1): 121 kB

Downloaded from central: https://repo.maven.apache.org/maven2/junit/junit/3.8.1/junit-3.8.1.jar (121 kB at 5.3 MB/s)
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ hiring ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.3.4.pom
Progress (1): 790 B
Progress (1): 2.3 kB
Progress (1): 4.3 kB
Progress (1): 5.8 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.3.4.pom
(5.8 kB at 307 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components/34/maven-shared-components-
34.pom
```



Stage 4 — Nexus Artifactory completed successfully.

STAGE 5 — TOMCAT DEPLOYMENT

24. Configure Tomcat Deployment

Created Jenkins credential:

tomcat-credentials

The Jenkins deployment user was configured in Tomcat with the required Manager role.

The Jenkins **Deploy war/ear to a container** option was checked, but Tomcat 10.1 was not available.

Therefore, deployment was performed using the Tomcat Manager API.

Added:

Build Steps → Execute shell

```
curl -u "jenkins:<Tomcat-password>" \  
  --upload-file target/hiring.war \  
  "http://<EC2-Public-IP>:8082/manager/text/deploy?path=/hiring&update=true"
```

Deployment path

/hiring

Verification

Opened:

<http://<EC2-Public-IP>:8082/hiring/>

Application displayed:

WELCOME TO TECHIE HORIZON

← → ↻ Not secure 100.48.208.250:8080/job/Hiring-App-Pipeline/12/console

Jenkins / Hiring-App-Pipeline / #12

Status

Changes

Console Output

Delete build #12

Git Build Data

Edit build information

Previous Build

Console

Download Copy View as plain text

This log is too long to show here, 14 KB has been skipped — click to see the complete log

```
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ hiring ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.3.4.pom
Progress (1): 790 B
Progress (1): 2.3 kB
Progress (1): 4.3 kB
Progress (1): 5.8 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.3.4.pom
(5.8 kB at 254 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components/34/maven-shared-components-34.pom
Progress (1): 781 B
Progress (1): 2.3 kB
Progress (1): 4.6 kB
Progress (1): 5.1 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components/34/maven-shared-components-34.pom
(5.1 kB at 255 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.6/commons-io-2.6.pom
Progress (1): 782 B
Progress (1): 2.1 kB
Progress (1): 5.3 kB
```

← → ↻ Not secure 100.48.208.250:8080/job/Hiring-App-Pipeline/12/console

Jenkins / Hiring-App-Pipeline / #12

Downloading from nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml
Progress (1): 329 B

Downloaded from nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml (329 B at 10.0 kB/s)
Uploading to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml
Progress (1): 360 B

Uploaded to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml (360 B at 8.6 kB/s)
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.296 s
[INFO] Finished at: 2026-10-04T15:48:07Z
[INFO] -----
+ rm -f /var/lib/jenkins/workspace/Hiring-App-Pipeline/settings.xml
[Hiring-App-Pipeline] \$ /bin/sh -xe /tmp/jenkins8861895647462574765.sh
+ curl -u jenkins:Jenkins@123 --upload-file target/hiring.war 'http://100.48.208.250:8082/manager/text/deploy?path=/hiring&update=true'
 % Total % Received % Xferd Average Speed Time Time Current
 Dload Upload Total Spent Left Speed

 0 0 0 0 0 0 0 0 0 0
100 1750 0 52 100 1698 1050 34287 0
100 1750 0 52 100 1698 1049 34264 0
100 1750 0 52 100 1698 1048 34252 0

OK - Deployed application at context path [/hiring]
[Slack Notifications] found #11 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS

← → ↻ Not secure 100.48.208.250:8082/hiring/

Welcome to Techie Horizon

Observation

The WAR was successfully deployed to Tomcat 10.1 and the application was accessible.

Result

Stage 5 — Tomcat Deployment completed successfully.

STAGE 6 — SLACK NOTIFICATION

25. Configure Slack

Installed the Slack Notification plugin.

Created Jenkins credential:

slack-token

Configured Slack under:

Manage Jenkins → System → Slack Notifications

Workspace:

shaik

Credential:

slack-token

Default channel:

#jenkins

Configured the Slack application with Bot Token Scopes:

chat:write

chat:write.public

Generated the Bot User OAuth Token and stored it in the Jenkins credential.

Clicked:

Test Connection

Result:

SUCCESS

26. Configure Slack Notifications in Job

In:

Hiring-App-Pipeline → Configure → Post-build Actions

Added:

Slack Notifications

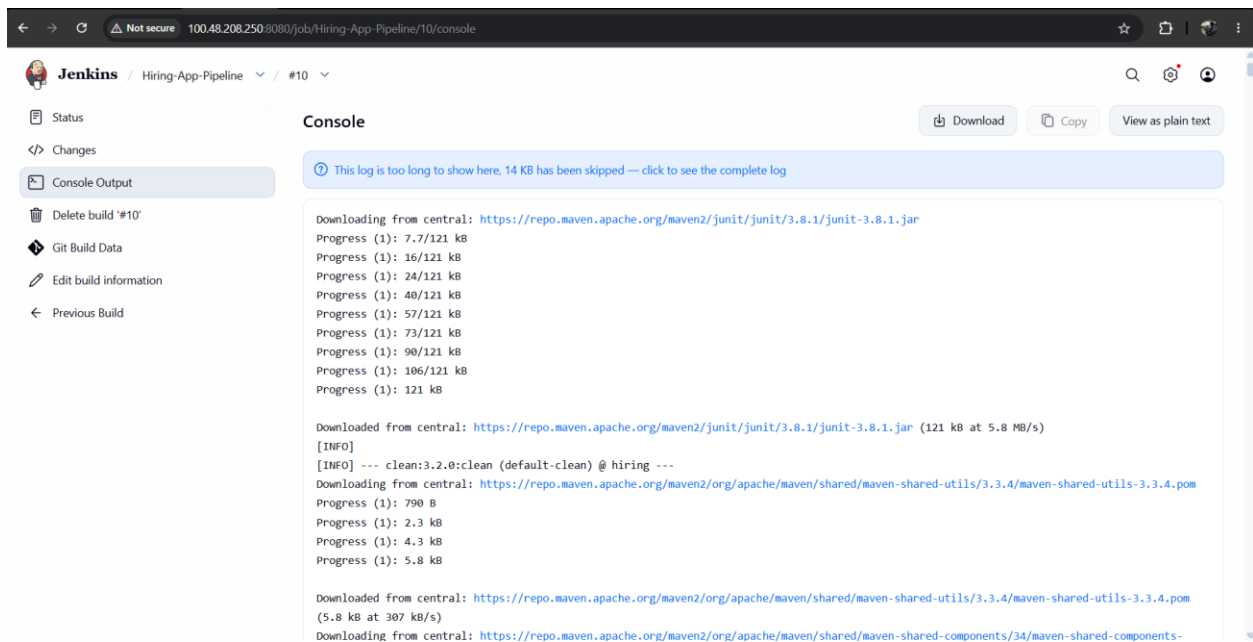
Enabled:

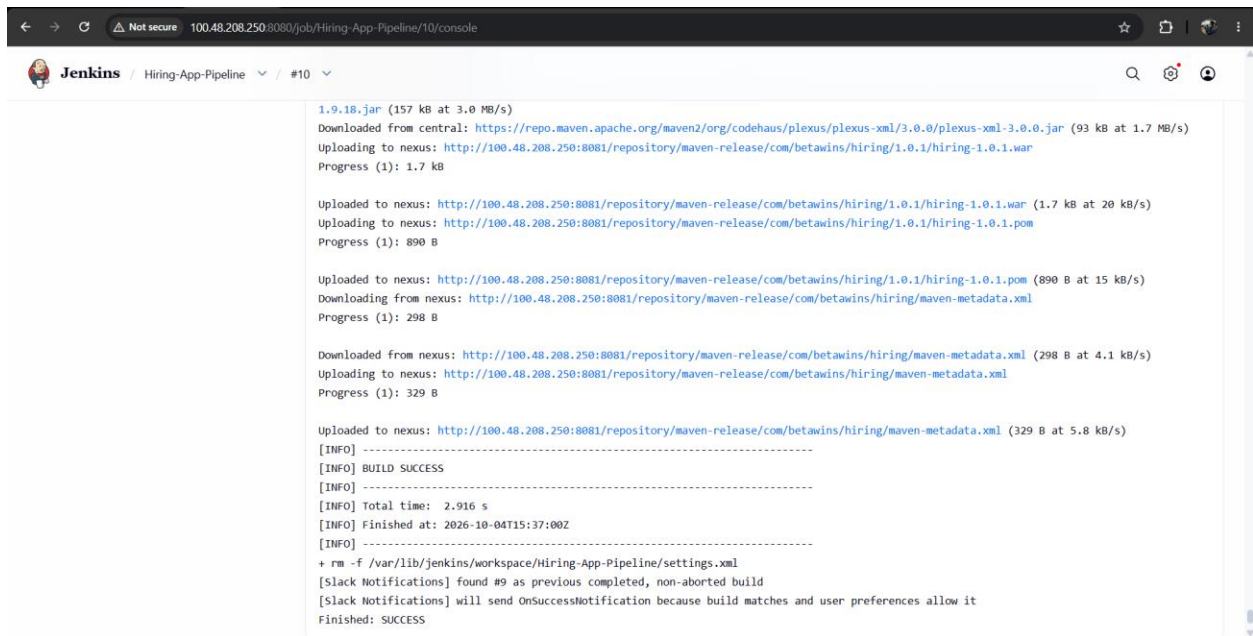
☒ Notify Success

☒ Notify Every Failure

Left the remaining notification options unchecked.

Saved the job.





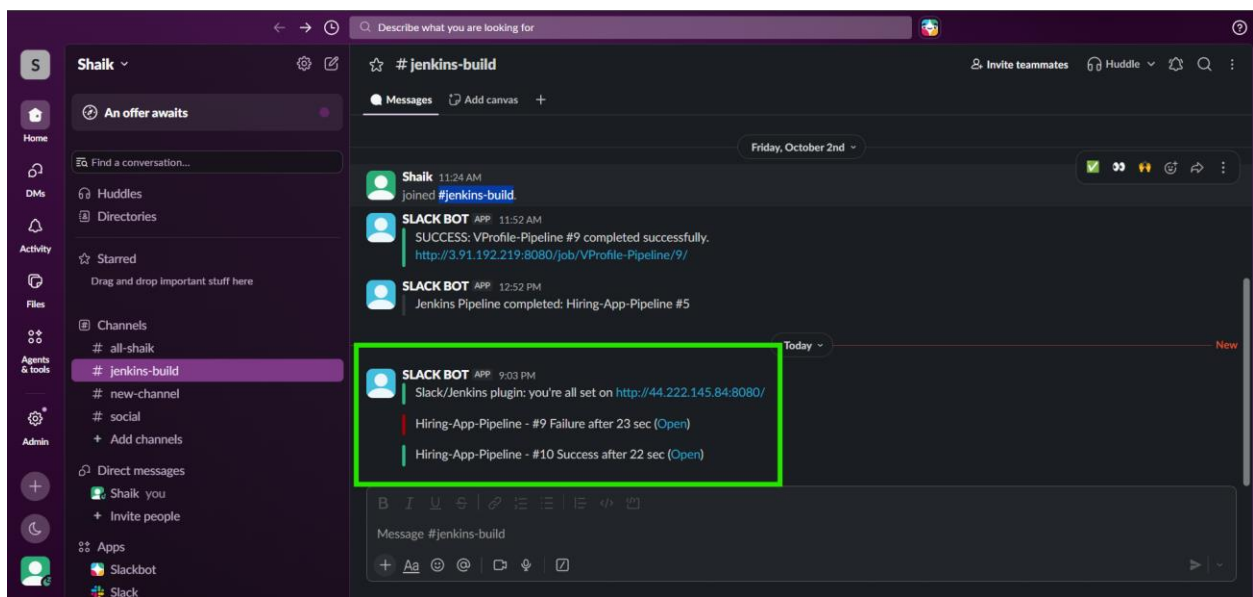
```
1.9.18.jar (157 kB at 3.0 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-xml/3.0.0/plexus-xml-3.0.0.jar (93 kB at 1.7 MB/s)
Uploading to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/1.0.1/hiring-1.0.1.war
Progress (1): 1.7 kB

Uploaded to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/1.0.1/hiring-1.0.1.war (1.7 kB at 20 kB/s)
Uploading to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/1.0.1/hiring-1.0.1.pom
Progress (1): 890 B

Uploaded to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/1.0.1/hiring-1.0.1.pom (890 B at 15 kB/s)
Downloading from nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml
Progress (1): 298 B

Downloaded from nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml (298 B at 4.1 kB/s)
Uploading to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml
Progress (1): 329 B

Uploaded to nexus: http://100.48.208.250:8081/repository/maven-release/com/betawins/hiring/maven-metadata.xml (329 B at 5.8 kB/s)
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.916 s
[INFO] Finished at: 2026-10-04T15:37:00Z
[INFO] -----
+ rm -f /var/lib/jenkins/workspace/Hiring-App-Pipeline/settings.xml
[Slack Notifications] found #9 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS
```



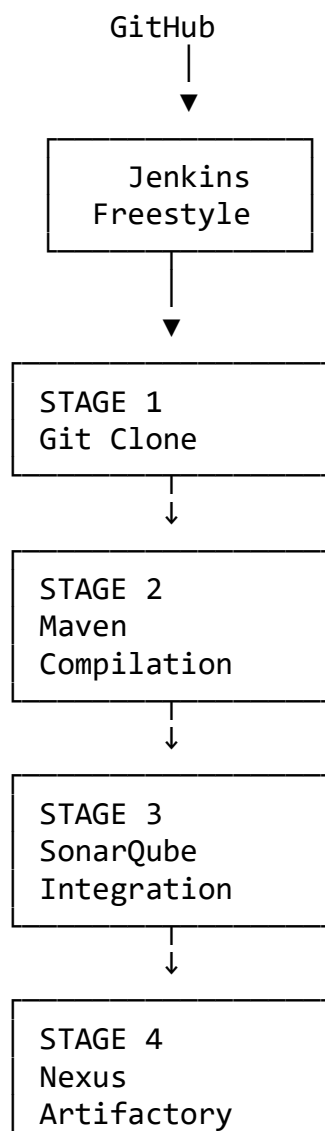
Observation

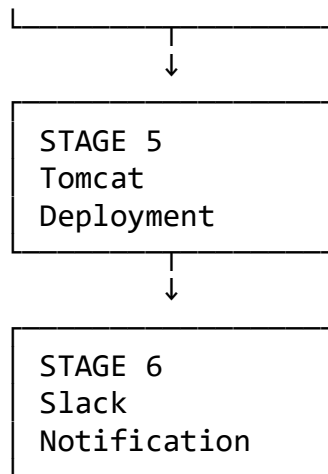
Jenkins successfully sent build status notifications to the Slack #jenkins channel.

Result

Stage 6 — Slack Notification completed successfully.

28. Final Pipeline Flow





29. Observation

The complete Freestyle Jenkins pipeline executed successfully.

- **Stage 1:** GitHub source code was cloned successfully.
- **Stage 2:** Maven compiled the application inside a Docker container.
- **Stage 3:** SonarQube successfully analyzed the source code.
- **Stage 4:** The WAR artifact was successfully uploaded to Nexus.
- **Stage 5:** The WAR was successfully deployed to Tomcat 10.1.
- **Stage 6:** Jenkins successfully sent build notifications to Slack.

30. Conclusion

The **Jenkins Freestyle CI/CD pipeline using Docker was successfully implemented end-to-end.**

The final automated workflow is:

Stage 1 → Git Clone
↓
Stage 2 → Maven Compilation
↓
Stage 3 → SonarQube Integration
↓

Stage 4 → Nexus Artifactory
↓
Stage 5 → Tomcat Deployment
↓
Stage 6 → Slack Notification

2) Setup a Jenkins CICD pipeline using Declarative pipeline using feature-1.1 branch.

Source Code: https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

Stages:

- **Git Clone**
- **SonarQube Integration**
- **Maven Compilation**
- **Nexus Artifactory**
- **Slack Notification**
- **Deploy On tomcat**

Objective

To create a Jenkins Declarative Pipeline using the feature-1.1 branch of the sabear_simplecutomerapp GitHub repository and automate:

Git Clone → Maven Compilation → SonarQube → Nexus → Tomcat → Slack

Stage 1 – Git Clone

Repository

https://github.com/betawins/sabear_simplecutomerapp.git

Branch

feature-1.1

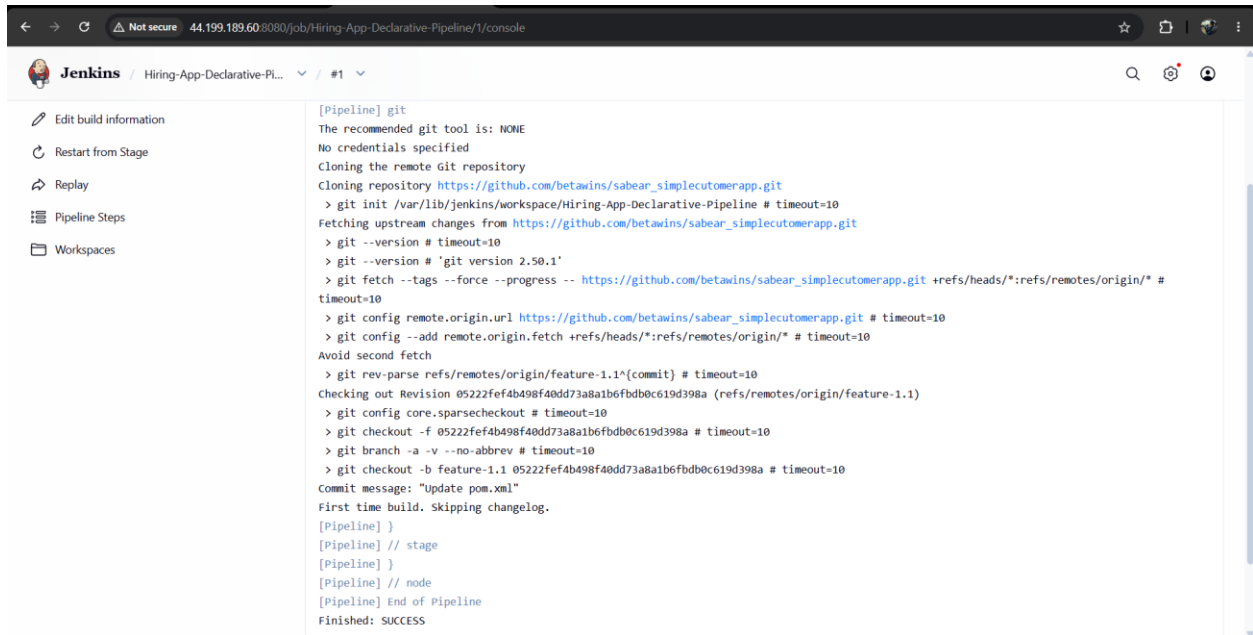
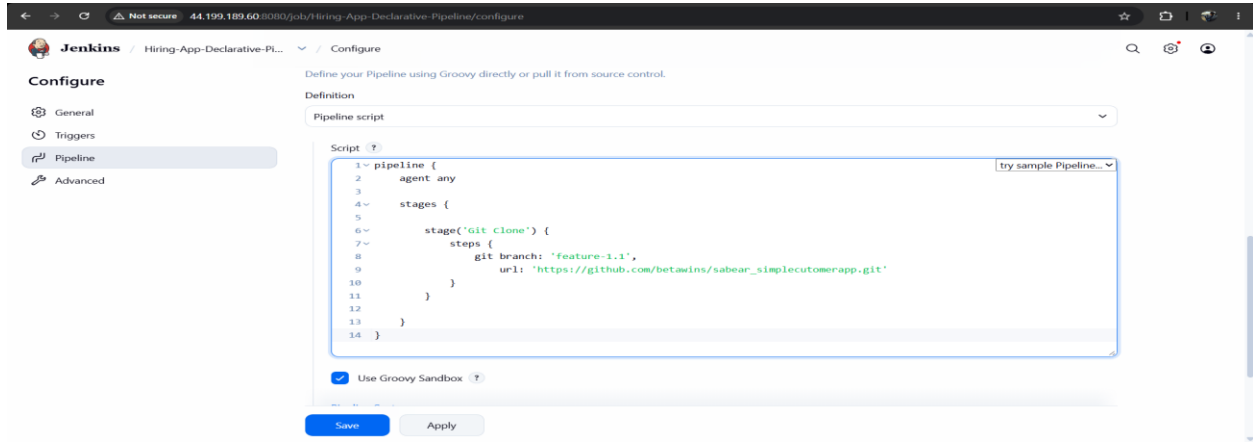
Pipeline Configuration

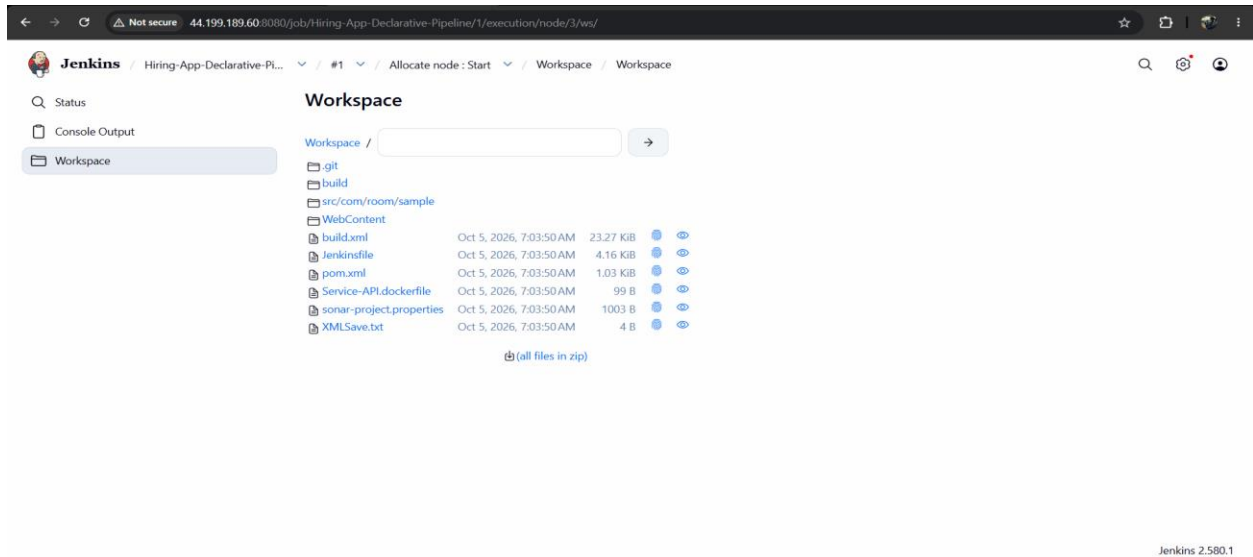
```
stage('Git Clone') {  
    steps {  
        git(  
            branch: 'feature-1.1',  
            url: 'https://github.com/betawins/sabear\_simplecutomerapp.git'  
        )  
    }  
}
```

Steps Used

1. Created a Jenkins **Pipeline** job named Hiring-App-Declarative-Pipeline.
2. Selected **Pipeline script** as the definition.
3. Configured the Git repository.

4. Selected the feature-1.1 branch.
5. Jenkins cloned the source code into the workspace.





Observation

The source code was successfully downloaded from the feature-1.1 branch into the Jenkins workspace.

Result

Git Clone — SUCCESS

Stage 2 – Maven Compilation

Purpose

To compile the Java source code and package the application as a WAR file using Maven inside a Docker container.

Docker Image Used

maven:3.9.9-eclipse-temurin-21

Pipeline Configuration

```
stage('Maven Compilation') {
    steps {
        sh '''
            docker run --rm \
                -v "$WORKSPACE:/app" \
                -w /app \
```

```

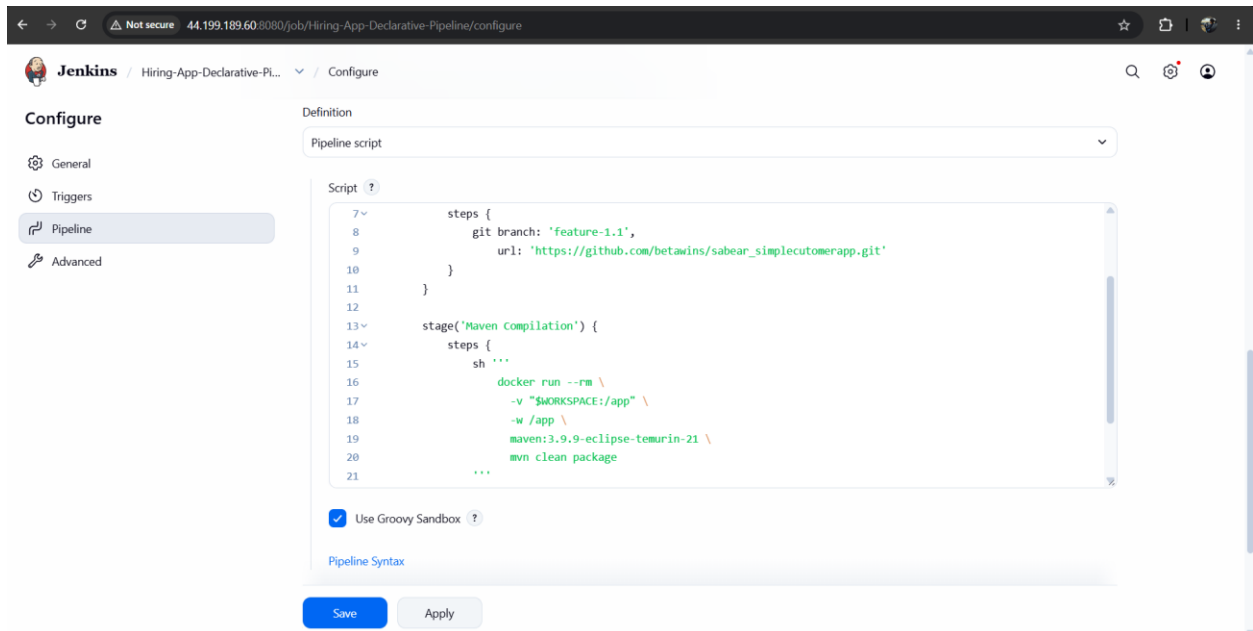
        maven:3.9.9-eclipse-temurin-21 \
        mvn clean package -DBUILD_NUMBER="$BUILD_NUMBER"
    """
}
}

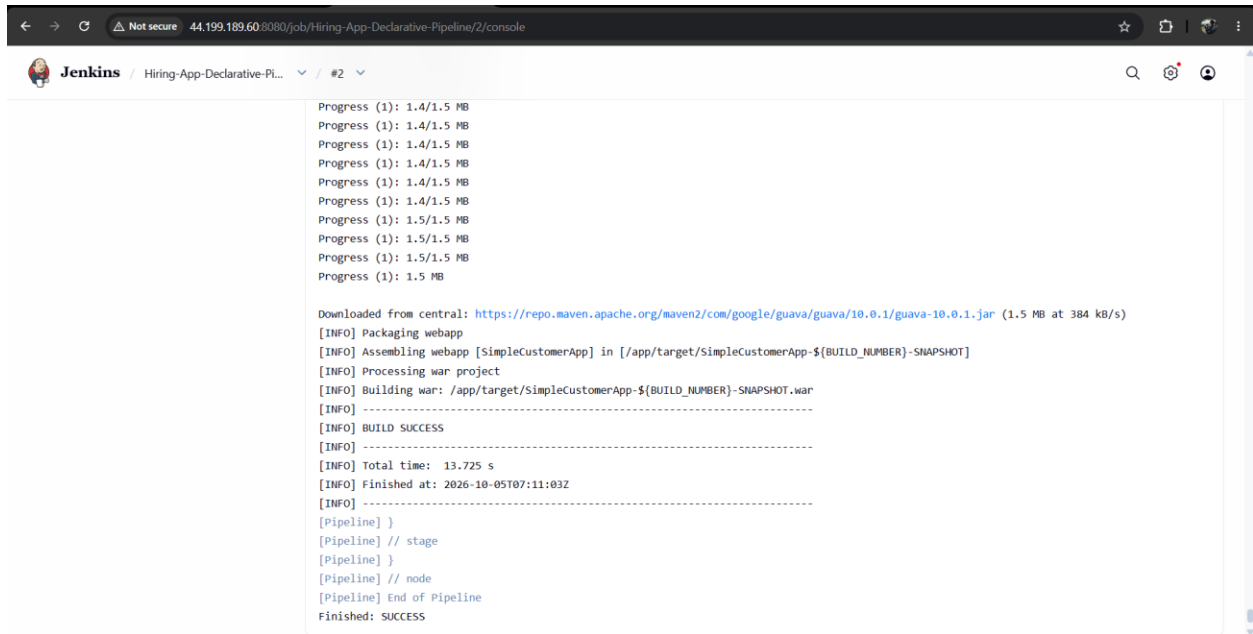
```

Steps Used

1. Jenkins mounted the workspace into the Maven Docker container.
2. Maven executed clean package.
3. Java source files were compiled.
4. The application was packaged as a WAR file.
5. The generated WAR file was:

target/SimpleCustomerApp-\${BUILD_NUMBER}-SNAPSHOT.war





The screenshot shows a Jenkins web interface with a console log for a build. The browser address bar indicates the URL is 44.199.189.60:8080/job/Hiring-App-Declarative-Pipeline/2/console. The Jenkins header shows the job name 'Hiring-App-Declarative-Pl...' and build number '#2'. The console output displays progress bars for downloading dependencies, Maven log messages for packaging, assembly, and war processing, and a final 'BUILD SUCCESS' message. Pipeline status messages at the bottom indicate the stage and node completion.

```
Progress (1): 1.4/1.5 MB
Progress (1): 1.4/1.5 MB
Progress (1): 1.4/1.5 MB
Progress (1): 1.4/1.5 MB
Progress (1): 1.4/1.5 MB
Progress (1): 1.4/1.5 MB
Progress (1): 1.5/1.5 MB
Progress (1): 1.5/1.5 MB
Progress (1): 1.5/1.5 MB
Progress (1): 1.5 MB

Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/10.0.1/guava-10.0.1-jar (1.5 MB at 384 kB/s)
[INFO] Packaging webapp
[INFO] Assembling webapp [SimpleCustomerApp] in [/app/target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT]
[INFO] Processing war project
[INFO] Building war: /app/target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 13.725 s
[INFO] Finished at: 2026-10-05T07:11:03Z
[INFO] -----
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Observation

Maven completed successfully and generated the WAR artifact.

The WAR was verified to contain:

WEB-INF/web.xml

Customer.jsp

Welcome.jsp

Result

Maven Compilation — SUCCESS

Stage 3 – SonarQube Integration

Purpose

To perform SonarQube analysis of the application source.

SonarQube

SonarQube Server: sonarqube

Project Key: SimpleCustomerApp-Declarative

Pipeline Configuration

```
stage('SonarQube Integration') {
    steps {
        withCredentials([
            string(
                credentialsId: 'sonarqube-token',
                variable: 'SONAR_TOKEN'
            )
        ]) {
            sh '''
                docker run --rm \
                --network devops-network \
                -v "$WORKSPACE:/usr/src" \
                -w /usr/src \
                sonarsource/sonar-scanner-cli:5 \
                sonar-scanner \
                -Dsonar.projectKey=SimpleCustomerApp-Declarative \
                -Dsonar.projectName=SimpleCustomerApp-Declarative \
                -Dsonar.sources=src \
                -Dsonar.exclusions=**/*.java \
                -Dsonar.host.url=http://sonarqube:9000 \
                -Dsonar.login="$SONAR_TOKEN"
            '''
        }
    }
}
```

Steps Used

1. Retrieved the SonarQube token from Jenkins Credentials.
2. Started the Sonar Scanner Docker container.
3. Connected the scanner to the devops-network.
4. Connected to the SonarQube server.
5. Used the project key SimpleCustomerApp-Declarative.
6. Completed the SonarQube analysis.

← → ↻ Not secure 44.199.189.60:8080/job/Hiring-App-Declarative-Pipeline/configure

Jenkins / Hiring-App-Declarative-Pi... / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Script ?

```
27 stage('SonarQube Integration') {
28   steps {
29     withCredentials([string(credentialsId: 'sonarqube-token', variable: 'SONAR_TOKEN')]) {
30       sh '''
31         docker run --rm \
32           --add-host=host.docker.internal:host-gateway \
33           -v "$WORKSPACE:/usr/src" \
34           -w /usr/src \
35           sonarsource/sonar-scanner-cli:5 \
36           -Dsonar.projectKey=SimpleCustomerApp \
37           -Dsonar.projectName=SimpleCustomerApp \
38           -Dsonar.sources=. \
39           -Dsonar.exclusions=**/*.java \
40           -Dsonar.host.url=http://host.docker.internal:9000 \
41           -Dsonar.login="$SONAR_TOKEN"
30
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

Advanced

Save Apply

← → ↻ Not secure 44.199.189.60:8080/job/Hiring-App-Declarative-Pipeline/12/console

Jenkins / Hiring-App-Declarative-Pi... / #12

Console Output

- Delete build '#12'
- Git Build Data
- Pipeline Overview
- Edit build information
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

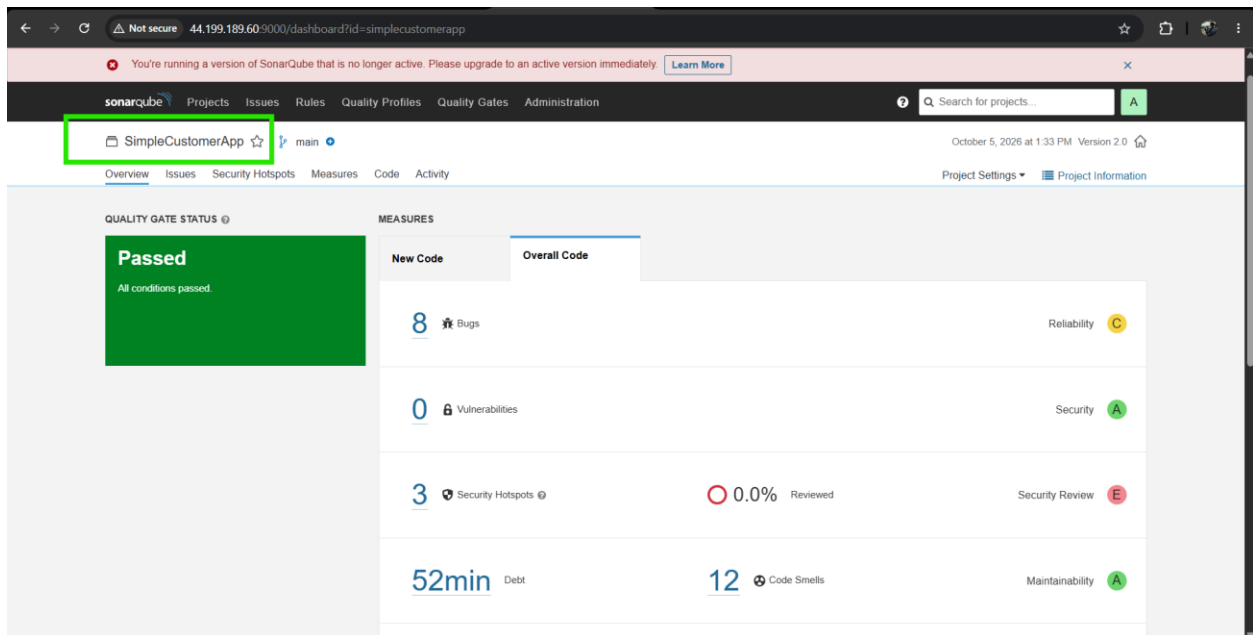
```
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components-34/maven-shared-components-34.pom
Progress (1): 781 B
Progress (1): 2.3 kB
Progress (1): 4.6 kB
Progress (1): 5.1 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components-34/maven-shared-components-34.pom (5.1 kB at 232 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/commons-io/commons-io-2.6/commons-io-2.6.pom
Progress (1): 782 B
Progress (1): 2.1 kB
Progress (1): 5.3 kB
Progress (1): 7.5 kB
Progress (1): 9.5 kB
Progress (1): 12 kB
Progress (1): 14 kB

Downloaded from central: https://repo.maven.apache.org/maven2/commons-io/commons-io-2.6/commons-io-2.6.pom (14 kB at 750 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.pom
Progress (1): 707 B
Progress (1): 1.9 kB
Progress (1): 3.1 kB
Progress (1): 4.5 kB
Progress (1): 5.9 kB
Progress (1): 7.5 kB
Progress (1): 11 kB
Progress (1): 14 kB
```

```
Jenkins / Hiring-App-Declarative-Pi... / #12

INFO: Sensor Zero Coverage Sensor
INFO: Sensor Zero Coverage Sensor (done) | time=2ms
INFO: SCM Publisher SCM provider for this project is: git
INFO: SCM Publisher 11 source files to be analyzed
INFO: SCM Publisher 11/11 source files have been analyzed (done) | time=264ms
INFO: CPD Executor Calculating CPD for 4 files
INFO: CPD Executor CPD calculation finished (done) | time=15ms
INFO: Analysis report generated in 87ms, dir size=241.4 kB
INFO: Analysis report compressed in 25ms, zip size=59.9 kB
INFO: Analysis report uploaded in 460ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: http://host.docker.internal:9000/dashboard?id=simplecustomerapp
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at http://host.docker.internal:9000/api/ce/task?id=AaELFs0U4tA5R0dEYVf
INFO: Analysis total time: 11.158 s
INFO: -----
INFO: EXECUTION SUCCESS
INFO: -----
INFO: Total time: 13.361s
INFO: Final Memory: 17M/64M
INFO: -----
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Observation

SonarQube analysis completed successfully and the project was submitted to the SonarQube server.

Result

SonarQube Integration — SUCCESS

Stage 4 – Nexus Artifactory

Purpose

To upload the generated WAR artifact to the Nexus Maven Snapshots repository.

Nexus Repository

maven-snapshots

Pipeline Configuration

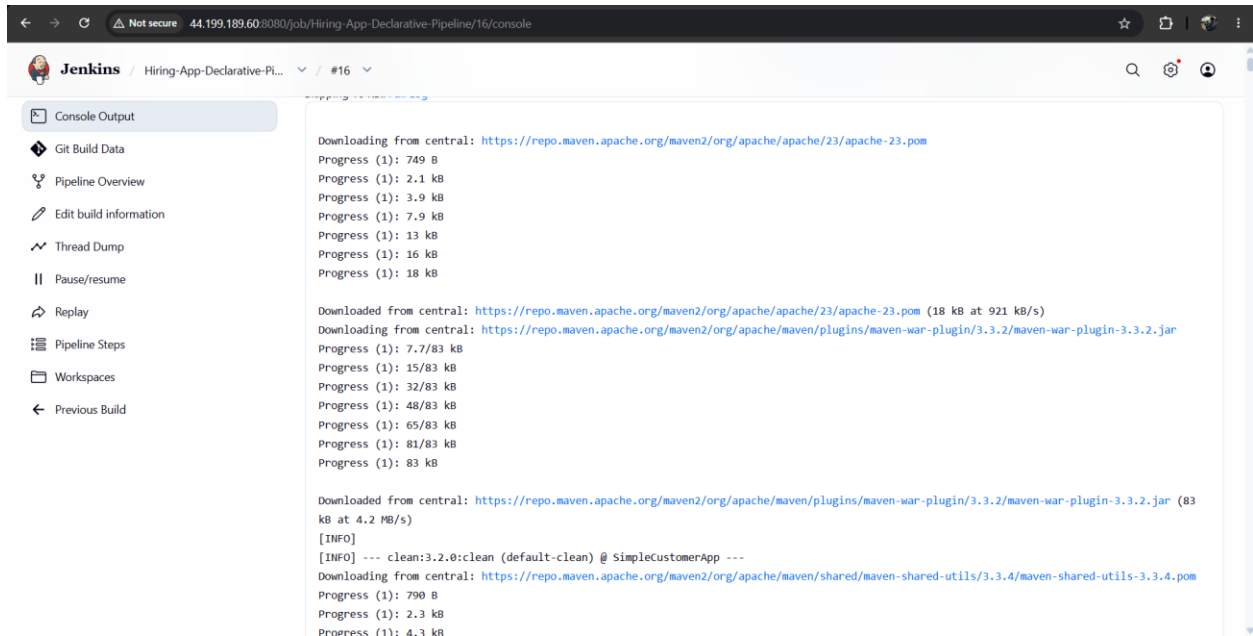
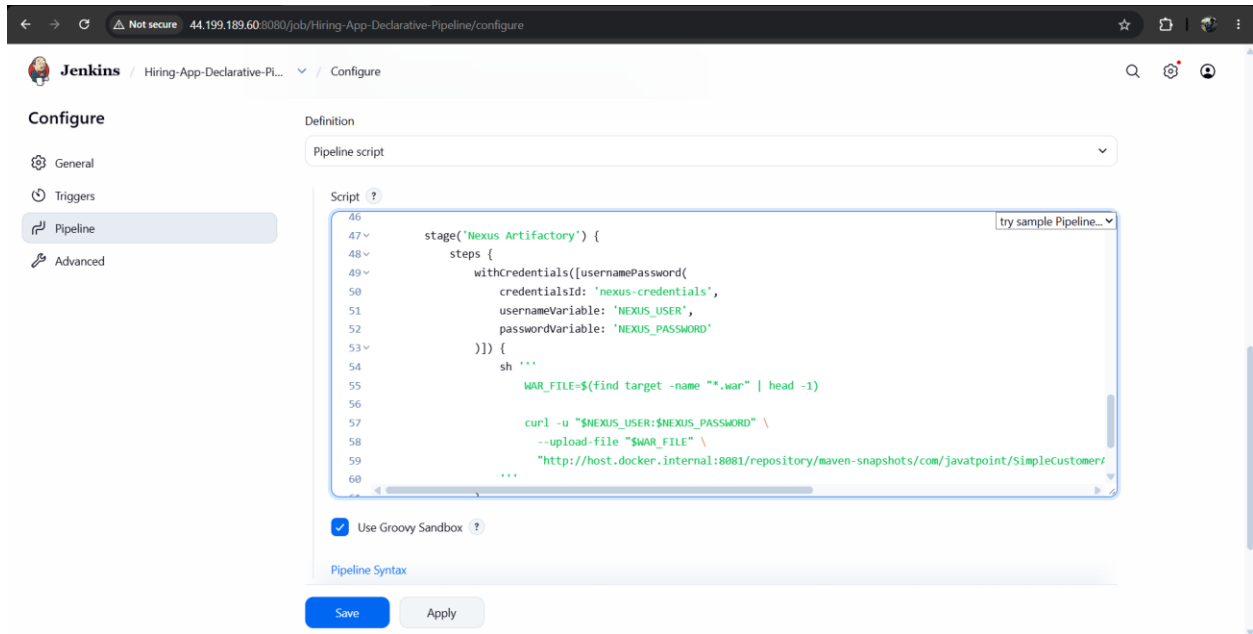
```
stage('Nexus Artifactory') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'nexus-credentials',
                usernameVariable: 'NEXUS_USER',
                passwordVariable: 'NEXUS_PASSWORD'
            )
        ]) {
            sh '''
                WAR_FILE="target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war"

                echo "WAR file: $WAR_FILE"

                curl -v \
                    -u "$NEXUS_USER:$NEXUS_PASSWORD" \
                    --upload-file "$WAR_FILE" \
                    "http://localhost:8081/repository/maven-snapshots/com/javatpoint/SimpleCustomerApp/\${BUILD\_NUMBER}-SNAPSHOT/SimpleCustomerApp-\${BUILD\_NUMBER}-SNAPSHOT.war"
            '''
        }
    }
}
```

Steps Used

1. Retrieved Nexus credentials from Jenkins Credentials.
2. Located the generated WAR file.
3. Connected to the Nexus Maven Snapshots repository.
4. Uploaded the WAR artifact using curl.
5. Nexus accepted the artifact.



The top screenshot shows a Jenkins console output for a pipeline named 'Hiring-App-Declarative-PL...'. The pipeline is at step #16. The output shows the following steps:

```
++ find target -type f -name '*.war'
++ head -1
+ WAR_FILE=target/SimpleCustomerApp-16-SNAPSHOT.war
++ basename target/SimpleCustomerApp-16-SNAPSHOT.war
+ WAR_NAME=SimpleCustomerApp-16-SNAPSHOT.war
+ echo 'WAR file found: SimpleCustomerApp-16-SNAPSHOT.war'
WAR file found: SimpleCustomerApp-16-SNAPSHOT.war
+ echo 'Uploading WAR to Nexus...'
Uploading WAR to Nexus...
+ curl -f -u admin:**** --upload-file target/SimpleCustomerApp-16-SNAPSHOT.war http://localhost:8081/repository/maven-
snapshots/com/javatpoint/SimpleCustomerApp-16-SNAPSHOT/SimpleCustomerApp-16-SNAPSHOT.war
```

% Total	% Received	% Xferd	Average	Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left	Speed
0	0	0	0	0	0	0	0	0
100	1422	0	0	100	1422	0	5370	0
100	1422	0	0	100	1422	0	5368	0
100	1422	0	0	100	1422	0	5367	0

```
+ echo 'Nexus upload completed successfully.'
Nexus upload completed successfully.
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

The bottom screenshot shows the Sonatype Nexus repository interface. The left sidebar shows the 'Browse' view for the 'maven-snapshots' repository. The main content area shows the directory structure:

- com
 - javatpoint
 - SimpleCustomerApp
 - 16-SNAPSHOT
 - SimpleCustomerApp-16-SNAPSHOT.war

Observation

The WAR artifact was successfully uploaded to Nexus.

Result

Nexus Artifactory — SUCCESS

Stage 5 – Tomcat Deployment

Purpose

To deploy the generated WAR application to the Tomcat server.

Tomcat

Tomcat: 9

Port: 8082

Application Context: /hiring

Pipeline Configuration

```
stage('Tomcat Deployment') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'tomcat-credentials',
                usernameVariable: 'TOMCAT_USER',
                passwordVariable: 'TOMCAT_PASSWORD'
            )
        ]) {
            sh '''
                WAR_FILE="target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war"

                echo "Deploying: $WAR_FILE"

                curl -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
                    "http://localhost:8082/manager/text/deploy?path=/hiring&update=true" \
                    --upload-file "$WAR_FILE"
            '''
        }
    }
}
```

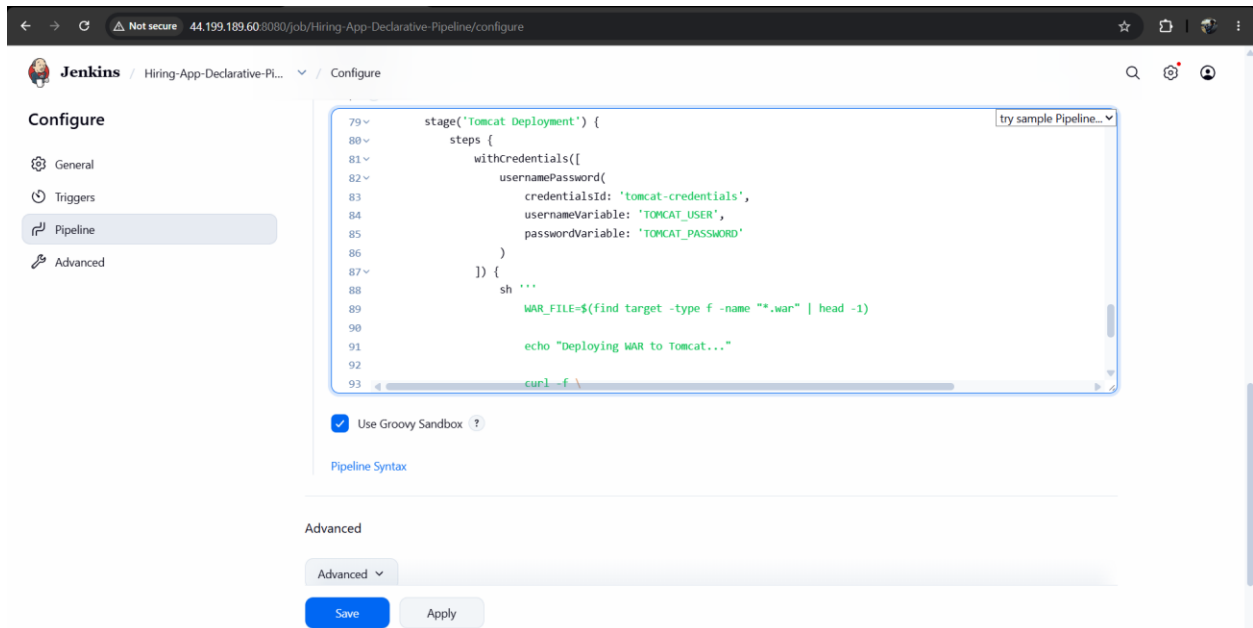
Steps Used

1. Retrieved Tomcat credentials from Jenkins Credentials.
2. Located the generated WAR file.
3. Connected to the Tomcat Manager API.
4. Deployed the WAR using the /hiring context path.
5. Restarted/loaded the application in Tomcat.

6. Verified the application through the browser.

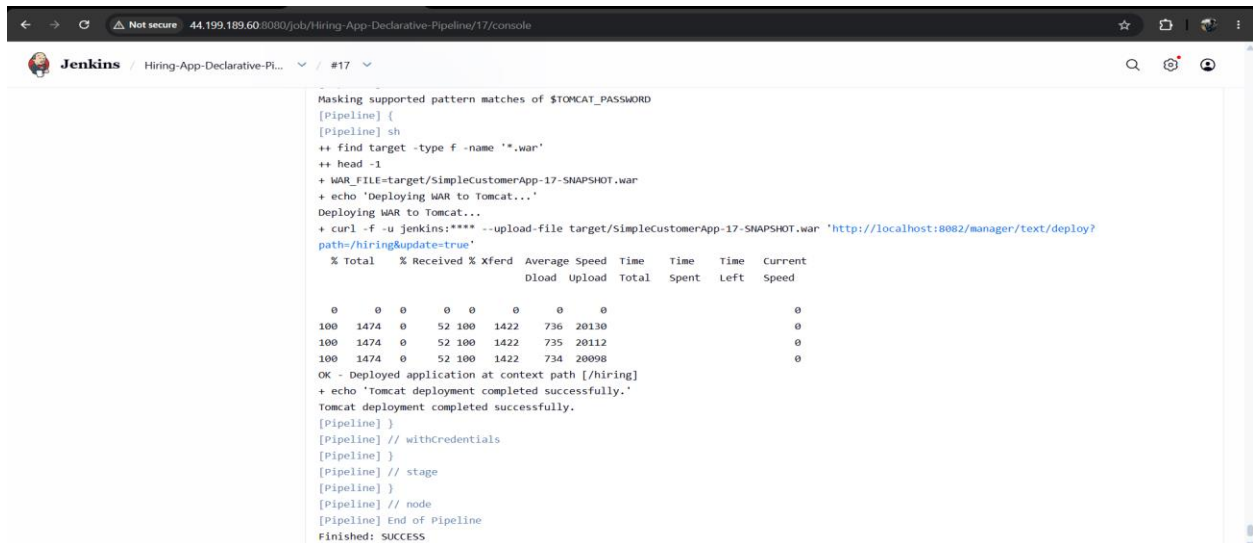
Application URL

<http://44.199.189.60:8082/hiring/Customer.jsp>



The screenshot shows the Jenkins 'Configure' page for a pipeline named 'Hiring-App-Declarative-Pipeline'. The 'Pipeline' tab is selected in the left sidebar. The main area displays a Groovy script for a stage named 'Tomcat Deployment'. The script includes steps for setting credentials and running a shell command to find and deploy a WAR file. Below the script, there is a 'Use Groovy Sandbox' checkbox which is checked, and a 'Pipeline Syntax' link. At the bottom, there is an 'Advanced' section with a 'Save' button and an 'Apply' button.

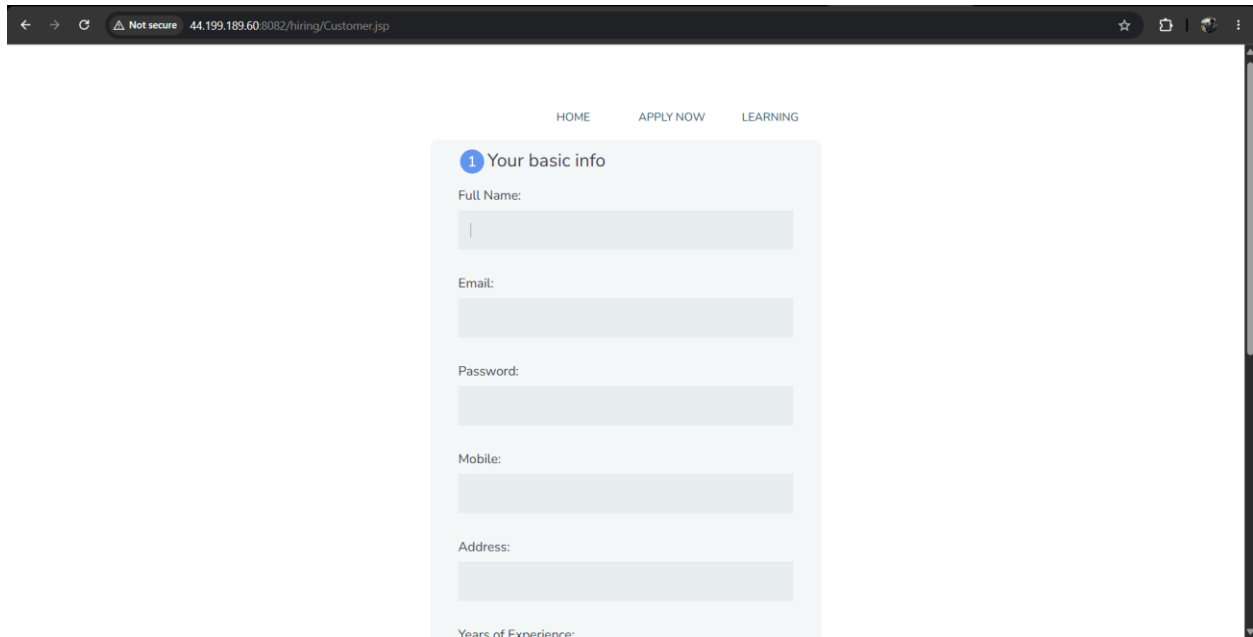
```
79 stage('Tomcat Deployment') {
80     steps {
81         withCredentials([
82             usernamePassword(
83                 credentialsId: 'tomcat-credentials',
84                 usernameVariable: 'TOMCAT_USER',
85                 passwordVariable: 'TOMCAT_PASSWORD'
86             )
87         ]) {
88             sh '''
89                 WAR_FILE=$(find target -type f -name "*.war" | head -1)
90
91                 echo "Deploying WAR to Tomcat..."
92
93                 curl -f \
```



The screenshot shows the Jenkins console output for the 'Hiring-App-Declarative-Pipeline' at step #17. The output displays the execution of the pipeline script, including the deployment of the WAR file to the Tomcat server. A table shows the progress of the file upload, and the final output indicates that the deployment was successful.

```
Masking supported pattern matches of $TOMCAT_PASSWORD
[Pipeline] {
[Pipeline] sh
++ find target -type f -name '*.war'
++ head -1
+ WAR_FILE=target/SimpleCustomerApp-17-SNAPSHOT.war
+ echo 'Deploying WAR to Tomcat...'
Deploying WAR to Tomcat...
+ curl -f -u jenkins:**** --upload-file target/SimpleCustomerApp-17-SNAPSHOT.war 'http://localhost:8082/manager/text/deploy?
path=/hiring&update=true'
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed

  0     0    0     0     0     0     0     0      0
100  1474    0    52 100  1422   736  20130      0
100  1474    0    52 100  1422   735  20112      0
100  1474    0    52 100  1422   734  20098      0
OK - Deployed application at context path [/hiring]
+ echo 'Tomcat deployment completed successfully.'
Tomcat deployment completed successfully.
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Observation

The application returned:

HTTP/1.1 200

The Customer.jsp page was successfully displayed in the browser.

Result

Tomcat Deployment — SUCCESS

Stage 6 – Slack Notification

Purpose

To send the Jenkins pipeline build status to the Slack channel.

Slack Channel

jenkins-build

Pipeline Configuration

```
stage('Slack Notification') {  
    steps {  
        slackSend(  
            channel: 'jenkins-build',  
            color: 'good',  
            message: "Jenkins Pipeline SUCCESS: ${env.JOB_NAME}  
#${env.BUILD_NUMBER}"  
        )  
    }  
}
```

Failure Notification

```
post {  
    failure {  
        slackSend(  
            channel: 'jenkins-build',  
            color: 'danger',  
            message: "Jenkins Pipeline FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"  
        )  
    }  
}
```

Steps Used

1. Configured the existing Slack integration in Jenkins.
2. Used the jenkins-build Slack channel.
3. Added a success notification after the pipeline stages.
4. Added a failure notification using the post section.
5. Jenkins sends the build status to Slack.

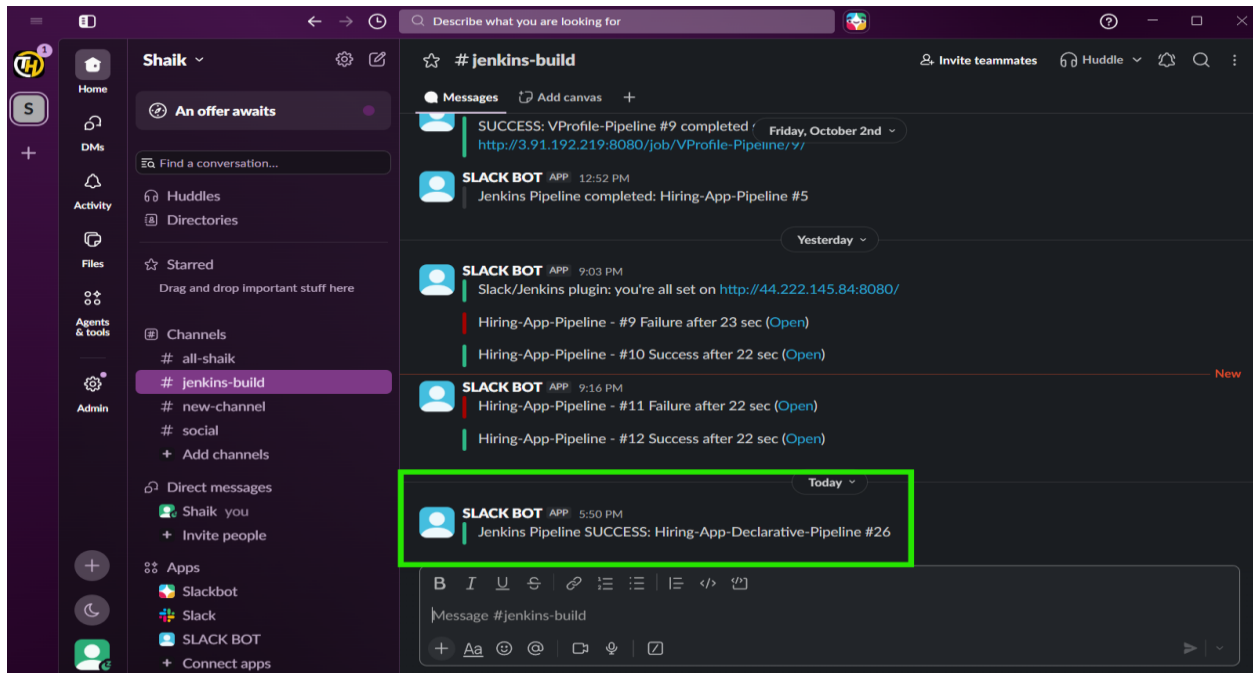
```
Jenkins / Hiring-App-Declarative-Pipeline/26/console

+ WAR_FILE=target/SimpleCustomerApp-26-SNAPSHOT.war
+ echo 'Deploying: target/SimpleCustomerApp-26-SNAPSHOT.war'
Deploying: target/SimpleCustomerApp-26-SNAPSHOT.war
+ curl -u jenkins:**** 'http://localhost:8080/manager/text/deploy?path=/hiring&update=true' --upload-file target/SimpleCustomerApp-26-SNAPSHOT.war

% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed

  0     0    0     0     0     0      0      0      0      0
100  1474    0    52 100  1422   981  26830      0      0
100  1474    0    52 100  1422   979  26795      0      0
100  1474    0    52 100  1422   978  26767      0      0

OK - Deployed application at context path [/hiring]
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Slack Notification)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: shaik, channel: jenkins-build, color: good, botuser: true,
tokenCredentialId: slack-token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Observation

The Jenkins pipeline build status was sent to the configured Slack channel.

Result

Slack Notification — SUCCESS

Observation

- The Jenkins Declarative Pipeline successfully executed all required stages sequentially.
- The source code was cloned from the feature-1.1 branch.
- Maven successfully compiled the application and generated the WAR artifact.
- The WAR file was verified to contain Customer.jsp, Welcome.jsp, and WEB-INF/web.xml.
- SonarQube successfully completed the configured source analysis.
- The generated WAR artifact was successfully uploaded to Nexus.
- The application was successfully deployed to Tomcat under the /hiring context.
- The deployed application returned **HTTP 200 OK** when accessing Customer.jsp.
- Slack notification was configured to send the pipeline status to the jenkins-build channel.

Troubleshooting

1. Maven POM Structure Issue

Problem:

Maven reported a malformed POM because warSourceDirectory was placed directly under the <build> element.

Solution:

Moved warSourceDirectory inside the maven-war-plugin <configuration> section.

Correct configuration:

```
<configuration>  
  <failOnMissingWebXml>false</failOnMissingWebXml>  
  <warSourceDirectory>WebContent</warSourceDirectory>  
</configuration>
```

2. POM File Structure Issue

Problem:

The pom.xml was accidentally left starting with the <build> element instead of the root <project> element.

Solution:

Restored the complete Maven POM with:

```
<project>
  ...
  <build>
    ...
  </build>
</project>
```

After correction, Maven completed successfully.

3. WAR Did Not Initially Contain Web Resources

Problem:

The WAR initially contained Java classes but did not contain the JSP files and web.xml. As a result, the Tomcat application returned **404**.

Solution:

Configured:

```
<sourceDirectory>src</sourceDirectory>
```

and:

```
<warSourceDirectory>WebContent</warSourceDirectory>
```

After rebuilding, the WAR contained:

```
WEB-INF/web.xml
Customer.jsp
Welcome.jsp
```


4. Tomcat Application Verification

Problem:

The /hiring/ root URL was not the correct endpoint for verifying the application.

Solution:

Verified the actual JSP page directly:

<http://44.199.189.60:8082/hiring/Customer.jsp>

The application returned:

HTTP/1.1 200

This confirmed successful Tomcat deployment.

Conclusion

The Jenkins Declarative Pipeline was successfully implemented using the feature-1.1 branch.

The complete automated flow was:

GitHub
↓
Git Clone
↓
Maven Compilation
↓
SonarQube
↓
Nexus Artifactory
↓
Tomcat Deployment
↓
Slack Notification

The application was successfully packaged as a WAR, stored in Nexus, deployed to Tomcat, and verified through the browser with **HTTP 200 OK**.

3) Setup a Jenkins CICD pipeline using Scripted pipeline using feature-1.1 branch.

Source Code: https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

Stages:

- Git Clone
- SonarQube Integration
- Maven Compilation
- Nexus Artifactory
- Slack Notification
- Deploy On tomcat

1. Objective

To create a Jenkins **Scripted Pipeline** for the sabear_simplecutomerapp project and automate the following stages:

Git Clone



Maven Compilation



SonarQube Integration



Nexus Artifactory



Deploy On Tomcat



Slack Notification

2. Prerequisites

The following were already available from the previous Jenkins setup:

- Jenkins installed on Amazon Linux EC2
- Jenkins running on port 8080
- Docker installed
- Jenkins user configured to use Docker
- Git installed
- SonarQube running in Docker
- Nexus running in Docker
- Tomcat running in Docker
- Jenkins credentials configured for:
 - SonarQube
 - Nexus
 - Tomcat
 - Slack
- Required Jenkins plugins installed

3. Create Jenkins Scripted Pipeline Job

Step 1: Open Jenkins

Open Jenkins using:

<http://44.199.189.60:8080>

Step 2: Create a new job

1. Click **New Item**.
2. Enter the job name:
Hiring-App-Scripted-Pipeline
3. Select **Pipeline**.
4. Click **OK**.

Step 3: Configure Pipeline

Go to the **Pipeline** section.

Select:

Definition → Pipeline script

The pipeline was written directly in the Jenkins Pipeline Script section.

Stage 1 — Git Clone

The first stage was used to download the application source code from GitHub.

Repository:

https://github.com/betawins/sabear_simplecutomerapp.git

Branch:

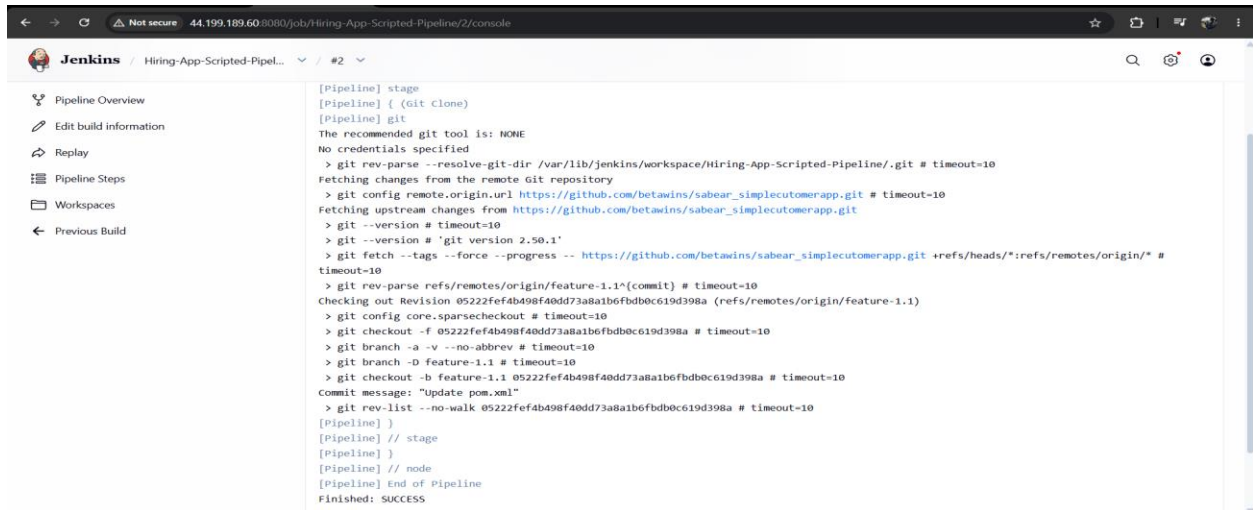
feature-1.1

Script used

```
stage('Git Clone') {  
    git(  
        branch: 'feature-1.1',  
        url: 'https://github.com/betawins/sabear_simplecutomerapp.git'  
    )  
}
```

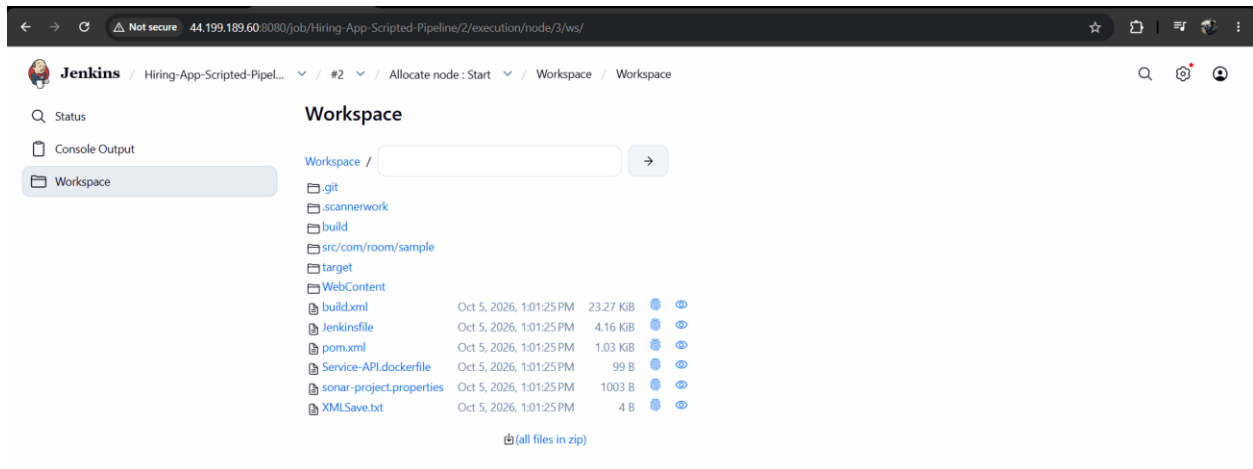
What happened

Jenkins connected to the GitHub repository and checked out the feature-1.1 branch into the Jenkins workspace.



The screenshot shows the Jenkins console output for a pipeline stage. The left sidebar contains links: Pipeline Overview, Edit build information, Replay, Pipeline Steps, Workspaces, and Previous Build. The main area displays the following log:

```
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Hiring-App-Scripted-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/sabear_simplecutterapp.git # timeout=10
Fetching upstream changes from https://github.com/betawins/sabear_simplecutterapp.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/sabear_simplecutterapp.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/feature-1.1^{commit} # timeout=10
checking out Revision 05222fef4b498f40dd73a8a1b6fbd0c619d398a (refs/remotes/origin/feature-1.1)
> git config core.sparsecheckout # timeout=10
> git checkout -f 05222fef4b498f40dd73a8a1b6fbd0c619d398a # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D feature-1.1 # timeout=10
> git checkout -b feature-1.1 05222fef4b498f40dd73a8a1b6fbd0c619d398a # timeout=10
commit message: "Update pom.xml"
> git rev-list --no-walk 05222fef4b498f40dd73a8a1b6fbd0c619d398a # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Observation

The Git clone stage completed successfully and the source code was available in:

/var/lib/jenkins/workspace/Hiring-App-Scripted-Pipeline

Stage 2 — Maven Compilation

The application was an older Java web project.

The project structure contained:

```
src/
WebContent/
pom.xml
```

The Java source files were under:

src/

and the web files were under:

WebContent/

Because this was not the standard Maven directory structure, the POM was configured accordingly.

Step 5.1 — Create the POM

The pipeline created the required pom.xml using a shell command.

Important configuration:

```
<groupId>com.javatpoint</groupId>
<artifactId>SimpleCustomerApp</artifactId>
<packaging>war</packaging>
<version>1.0-SNAPSHOT</version>
```

The Java source directory was configured as:

```
<sourceDirectory>src</sourceDirectory>
```

The web application directory was configured as:

```
<warSourceDirectory>WebContent</warSourceDirectory>
```

The Maven WAR plugin was configured to generate the web application WAR file.

The required dependencies included:

```
javax.servlet-api
junit
```

Step 5.2 — Run Maven inside Docker

Maven was executed using a Docker container.

Command used:

```
docker run --rm \  
-v "$WORKSPACE:/usr/src" \  
-w /usr/src \  
maven:3.9-eclipse-temurin-8 \  
mvn clean package
```

Explanation

docker run

Runs Maven inside a Docker container.

--rm

Removes the Maven container after the build finishes.

-v "\$WORKSPACE:/usr/src"

Mounts the Jenkins workspace into the Docker container.

-w /usr/src

Sets /usr/src as the working directory.

maven:3.9-eclipse-temurin-8

Uses the Maven Docker image with Java 8.

mvn clean package

Cleans the previous build and creates the application package.

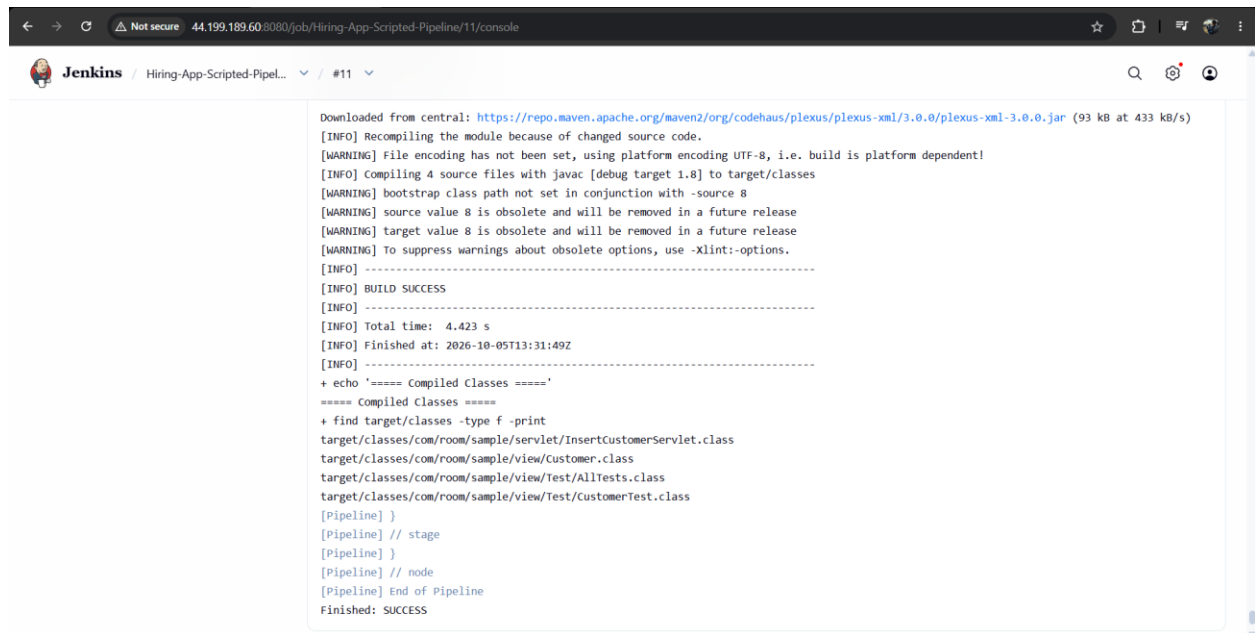
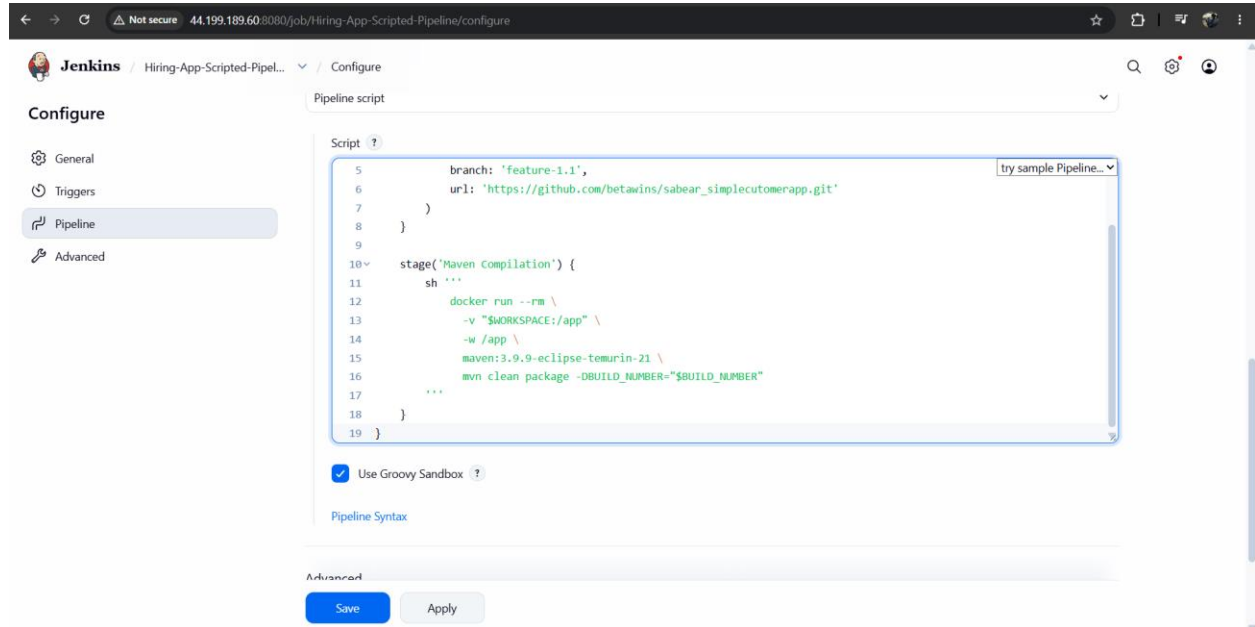
Step 5.3 — Verify WAR file

The pipeline checked the generated WAR file:

```
ls -lh target/*.war
```

The generated file was:

```
target/SimpleCustomerApp-1.0-SNAPSHOT.war
```



Observation

Maven successfully compiled the Java source code and generated the WAR file.

Stage 3 — SonarQube Integration

After Maven compilation, the source code was analyzed using SonarQube.

The SonarQube scanner was executed using Docker.

SonarQube configuration

Project Key: SimpleCustomerApp-Scripted
Project Name: SimpleCustomerApp-Scripted
Source Directory: src
Java Binaries: target/classes
SonarQube URL: <http://sonarqube:9000>

Script used

```
stage('SonarQube Integration') {  
    withCredentials([  
        string(  
            credentialsId: 'sonarqube-token',  
            variable: 'SONAR_TOKEN'  
        )  
    ]) {  
        sh '''  
            echo "===== SonarQube Analysis ====="  
  
            docker run --rm \  
                --network devops-network \  
                -v "$WORKSPACE:/usr/src" \  
                -w /usr/src \  
                sonarsource/sonar-scanner-cli:5 \  
                sonar-scanner \  
                -Dsonar.projectKey=SimpleCustomerApp-Scripted \  
                -Dsonar.projectName=SimpleCustomerApp-Scripted \  
                -Dsonar.sources=src \  
                -Dsonar.java.binaries=/usr/src/target/classes \  
                -Dsonar.host.url=http://sonarqube:9000 \  
                -Dsonar.login="$SONAR_TOKEN"  
            '''  
    }  
}
```

```
}  
}
```

Important configuration

The Docker container was connected to:

devops-network

This allowed the SonarQube scanner container to communicate with the SonarQube container using:

<http://sonarqube:9000>

The compiled Java classes were provided through:

-Dsonar.java.binaries=/usr/src/target/classes

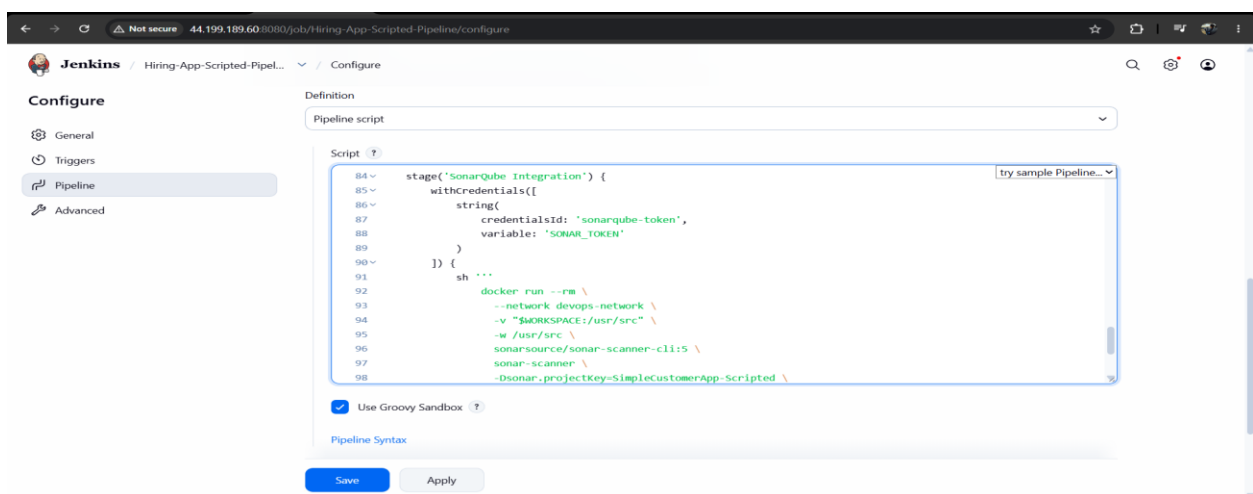
Result

SonarQube returned:

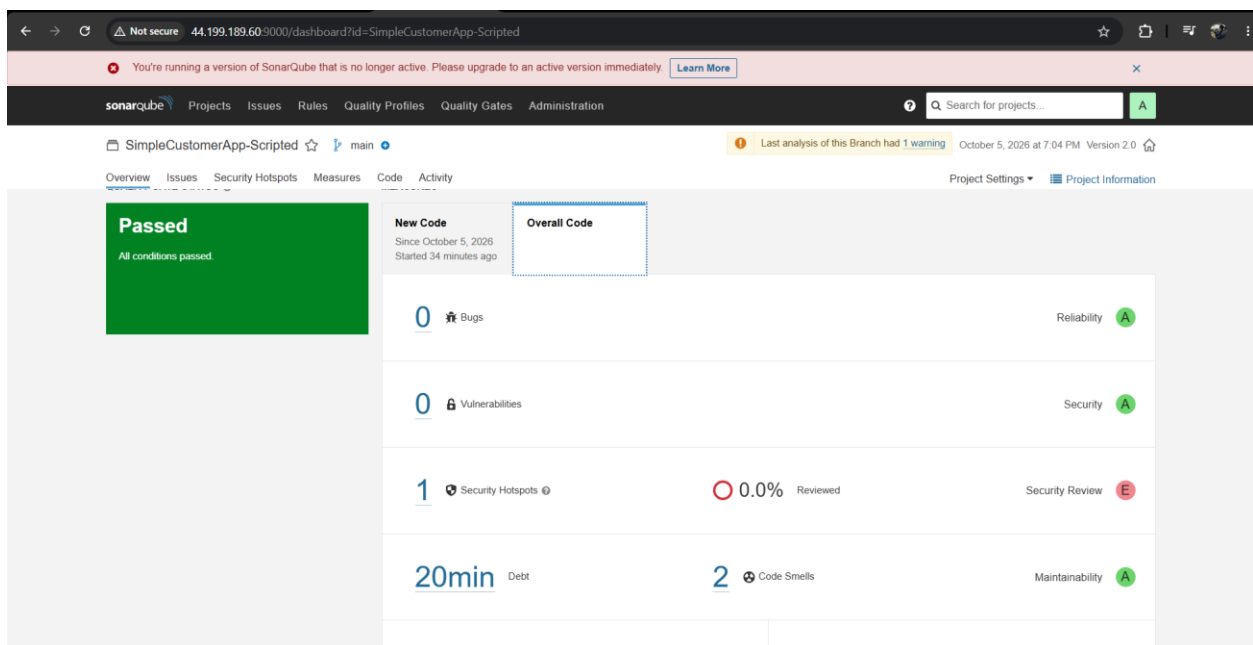
ANALYSIS SUCCESSFUL

EXECUTION SUCCESS

The analysis processed **4 Java source files**.



```
INFO: sensor java CPD BLOCK indexer (done) | time=31ms
INFO: SCM Publisher SCM provider for this project is: git
INFO: SCM Publisher 4 source files to be analyzed
INFO: SCM Publisher 4/4 source files have been analyzed (done) | time=214ms
INFO: CPD Executor 1 file had no CPD blocks
INFO: CPD Executor Calculating CPD for 3 files
INFO: CPD Executor CPD calculation finished (done) | time=8ms
INFO: Analysis report generated in 112ms, dir size=137.3 kB
INFO: Analysis report compressed in 17ms, zip size=25.6 kB
INFO: Analysis report uploaded in 18ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: http://sonarqube:9000/dashboard?id=SimpleCustomerApp-Scripted
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at http://sonarqube:9000/api/ce/task?id=AaEMRe44tA5RDdEYv-
INFO: Analysis total time: 6.033 s
INFO: -----
INFO: EXECUTION SUCCESS
INFO: -----
INFO: Total time: 8.105s
INFO: Final Memory: 21M/77M
INFO: -----
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Observation

The SonarQube analysis completed successfully and the project was updated in the SonarQube dashboard.

Stage 4 — Nexus Artifactory

After successful SonarQube analysis, the generated WAR file was uploaded to Nexus.

The Jenkins Nexus credentials were accessed using:

withCredentials

WAR file

target/SimpleCustomerApp-1.0-SNAPSHOT.war

Nexus repository

maven-snapshots

Script used

```
stage('Nexus Artifactory') {
    withCredentials([
        usernamePassword(
            credentialsId: 'nexus-credentials',
            usernameVariable: 'NEXUS_USER',
            passwordVariable: 'NEXUS_PASSWORD'
        )
    ]) {
        sh '''
            WAR_FILE="target/SimpleCustomerApp-1.0-SNAPSHOT.war"

            echo "===== Uploading WAR to Nexus ====="
            echo "WAR File: $WAR_FILE"

            curl -f\
                -u "$NEXUS_USER:$NEXUS_PASSWORD" \
                --upload-file "$WAR_FILE" \
                "http://localhost:8081/repository/maven-snapshots/com/javatpoint/SimpleCustomerApp/1.0-SNAPSHOT/SimpleCustomerApp-1.0-SNAPSHOT.war"
        '''
    }
}
```

What happened

Jenkins used curl to upload the generated WAR file to the Nexus Maven snapshots repository.

← → ↻ Not secure 44.199.189.60:8080/job/Hiring-App-Scripted-Pipeline/configure

Jenkins / Hiring-App-Scripted-Pipel... / Configure

Configure

Pipeline script

Script ?

```
108 stage('Nexus Artifactory') {
109     withCredentials([
110         usernamePassword(
111             credentialsId: 'nexus-credentials',
112             usernameVariable: 'NEXUS_USER',
113             passwordVariable: 'NEXUS_PASSWORD'
114         )
115     ]) {
116         sh '''
117             WAR_FILE=target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war
118
119             echo "===== Uploading WAR to Nexus ====="
120
121             curl -f \
122                 -u "$NEXUS_USER:$NEXUS_PASSWORD" \
```

try sample Pipeline...

☒ Use Groovy Sandbox ?

Pipeline Syntax

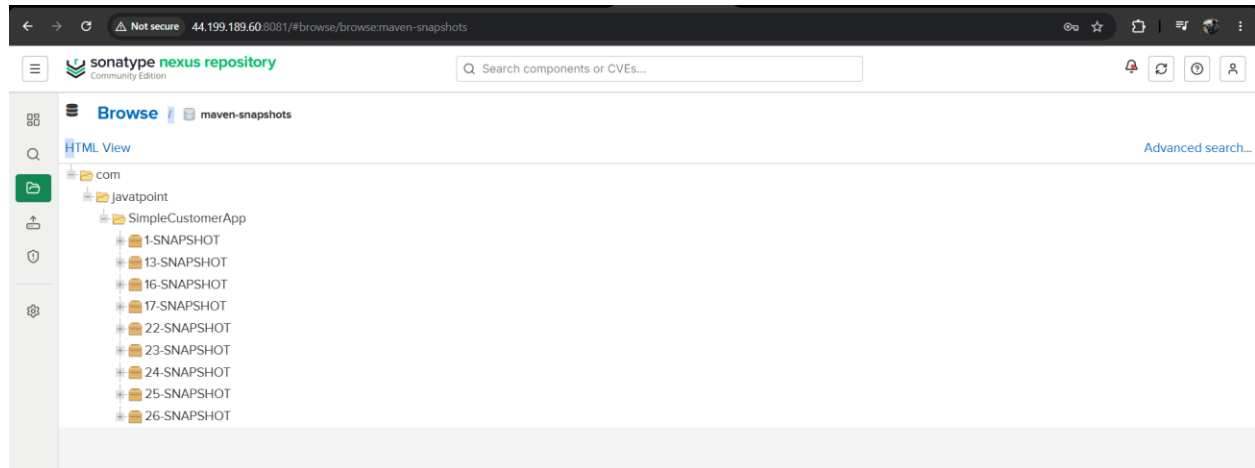
Save Apply

← → ↻ Not secure 44.199.189.60:8080/job/Hiring-App-Scripted-Pipeline/13/console

Jenkins / Hiring-App-Scripted-Pipel... / #13

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Nexus Artifactory)
[Pipeline] withCredentials
Masking supported pattern matches of $NEXUS_PASSWORD
[Pipeline] {
[Pipeline] sh
+ WAR_FILE=target/SimpleCustomerApp-13-SNAPSHOT.war
+ echo '===== Uploading WAR to Nexus ====='
===== Uploading WAR to Nexus =====
+ curl -f -u admin:**** --upload-file target/SimpleCustomerApp-13-SNAPSHOT.war http://localhost:8081/repository/maven-
snapshots/com/javatpoint/SimpleCustomerApp/13-SNAPSHOT/SimpleCustomerApp-13-SNAPSHOT.war
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed

  0     0    0     0  0     0     0     0  0
100  1.25M    0     0  100  1.25M    0  24.48M    0
100  1.25M    0     0  100  1.25M    0  24.45M    0
100  1.25M    0     0  100  1.25M    0  24.42M    0
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Observation

The WAR file was successfully uploaded to Nexus without any error.

Stage 5 — Deploy On Tomcat

After uploading the artifact to Nexus, the same generated WAR file was deployed to Tomcat.

Tomcat was running on:

localhost:8082

The Tomcat Manager API was used for deployment.

Script used

```
stage('Deploy On Tomcat') {
    withCredentials([
        usernamePassword(
            credentialsId: 'tomcat-credentials',
            usernameVariable: 'TOMCAT_USER',
            passwordVariable: 'TOMCAT_PASSWORD'
        )
    ]) {
        sh '''
            WAR_FILE="target/SimpleCustomerApp-1.0-SNAPSHOT.war"

            echo "==== Deploying WAR to Tomcat ====="
```

```
curl -f \
-u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
"http://localhost:8082/manager/text/deploy?path=/hiring&update=true" \
--upload-file "$WAR_FILE"

echo
echo "===== Checking Application ====="

curl -I \
"http://localhost:8082/hiring/Customer.jsp"
'''
}
}
```

Deployment context

The application was deployed with:

/hiring

Therefore, the application URL became:

<http://44.199.189.60:8082/hiring/Customer.jsp>

Deployment result

Tomcat returned:

OK - Deployed application at context path [/hiring]

The application was then checked using:

curl -I <http://localhost:8082/hiring/Customer.jsp>

The response was:

HTTP/1.1 200

← → ↻ Not secure 44.199.189.60:8080/job/Hiring-App-Scripted-Pipeline/configure

Jenkins / Hiring-App-Scripted-Pipel... / Configure

Configure

General
Triggers
Pipeline
Advanced

Pipeline script

Script ?

```
129 stage('Deploy On Tomcat') {  
130     withCredentials([  
131         usernamePassword(  
132             credentialsId: 'tomcat-credentials',  
133             usernameVariable: 'TOMCAT_USER',  
134             passwordVariable: 'TOMCAT_PASSWORD'  
135         )  
136     ]) {  
137         sh '''  
138             WAR_FILE=target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war  
139  
140             echo "==== Deploying WAR to Tomcat ====="  
141  
142             curl -f \\  
143                 -u "${TOMCAT_USER}:${TOMCAT_PASSWORD}" \
```

try sample Pipeline...

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

Save Apply

← → ↻ Not secure 44.199.189.60:8080/job/Hiring-App-Scripted-Pipeline/14/console

Jenkins / Hiring-App-Scripted-Pipel... / #14

```
100 1.25M 0 52 100 1.25M 610 14.72M 0  
OK - Deployed application at context path [/hiring]  
+ echo  
  
+ echo '==== Checking Application ====='  
==== Checking Application =====  
+ curl -I http://localhost:8082/hiring/customer.jsp  
% Total % Received % Xferd Average Speed Time Time Time Current  
 Dload Upload Total Spent Left Speed  
  
0 0 0 0 0 0 0 0 0  
0 4595 0 0 0 0 0 0 0  
0 4595 0 0 0 0 0 0 0  
0 4595 0 0 0 0 0 0 0  
HTTP/1.1 200  
Set-Cookie: JSESSIONID=D5B885F3EE762D1A5A93F505A0ACD0EB; Path=/hiring; HttpOnly  
Content-Type: text/html; charset=ISO-8859-1  
Content-Length: 4595  
Date: Mon, 05 Oct 2026 13:43:10 GMT  
  
[Pipeline] }  
[Pipeline] // withCredentials  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```


← → ↻ Not secure 44.199.189.60:8082/hiring/Customer.jsp

HOME APPLY NOW LEARNING

1 Your basic info

Full Name:

Email:

Password:

Mobile:

Address:

Years of Experience:

0 - 5 yrs

Observation

The application was successfully deployed to Tomcat and returned **HTTP 200**, confirming that the application was accessible.

Stage 6 — Slack Notification

The final stage was used to send the Jenkins pipeline completion notification to Slack.

Slack channel:

jenkins-build

Script used

```
stage('Slack Notification') {  
    slackSend(  
        channel: 'jenkins-build',  
        message: "Jenkins Pipeline completed: ${env.JOB_NAME} #${env.BUILD_NUMBER}"  
    )  
}
```

What happened

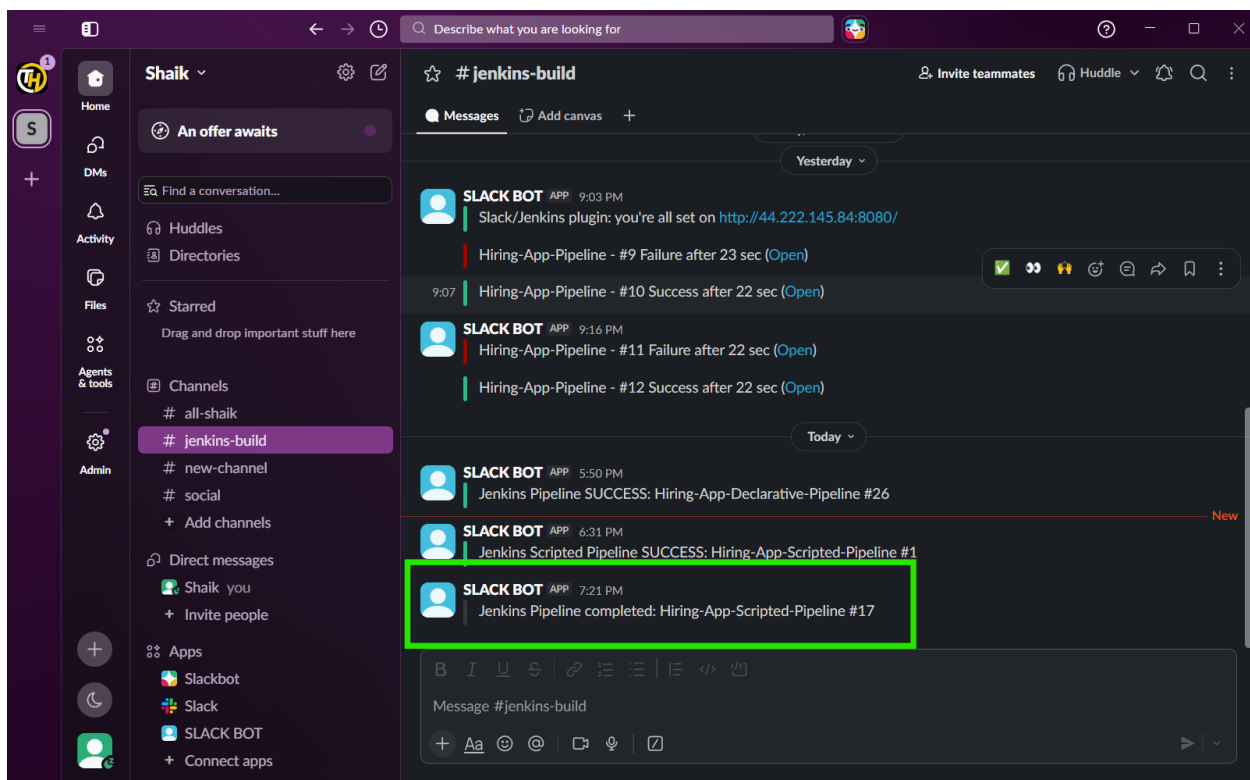
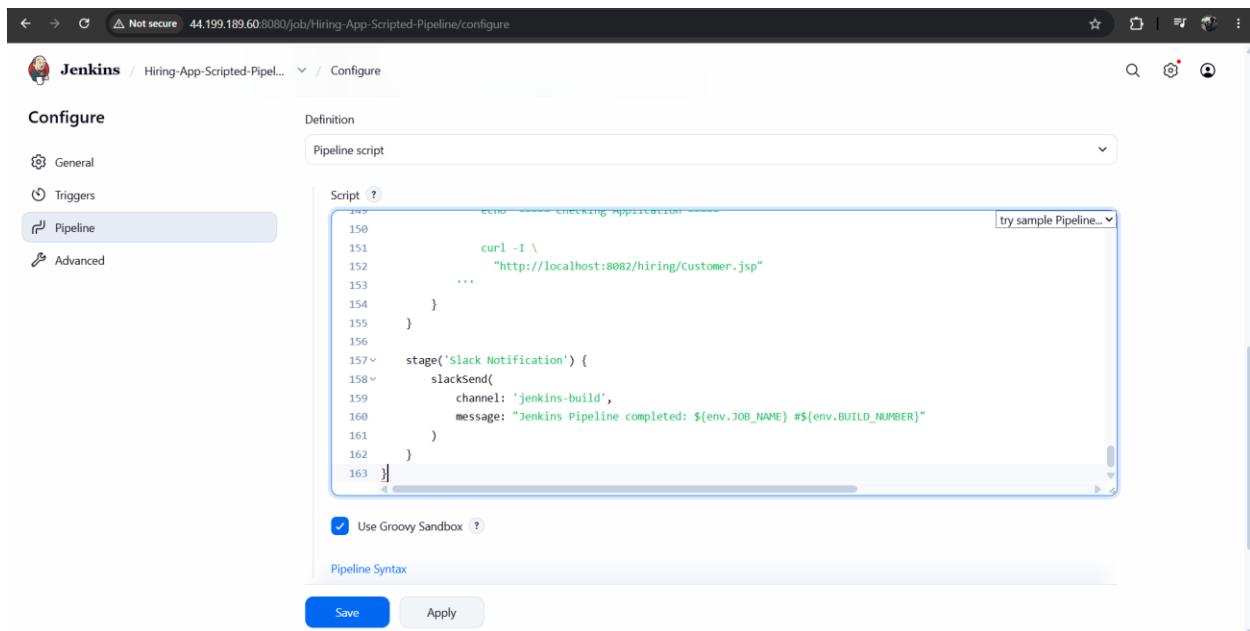
After all previous stages completed successfully, Jenkins sent a notification to the configured Slack channel.

The Slack configuration used the previously configured:

Workspace: shaik

Channel: jenkins-build

Credential: slack-token

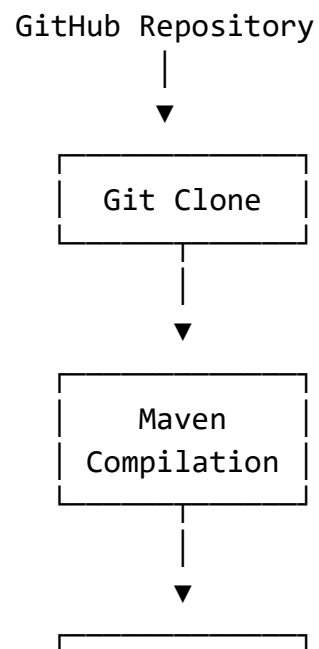


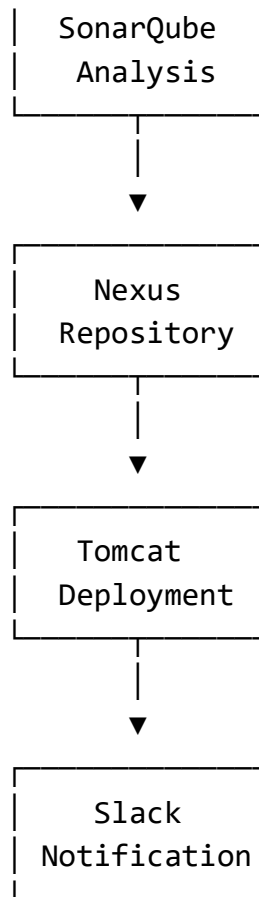
Observation

The Slack notification was successfully sent after pipeline completion.

10. Complete Pipeline Flow

The final Scripted Pipeline worked in the following order:





11. Final Verification

The Jenkins console showed:

Finished: SUCCESS

Tomcat deployment showed:

OK - Deployed application at context path [/hiring]

Application verification showed:

HTTP/1.1 200

SonarQube showed:

ANALYSIS SUCCESSFUL
EXECUTION SUCCESS

Nexus upload completed successfully.

Slack notification was sent to:

jenkins-build

12. Troubleshooting During Implementation

Issue 1 — Maven Version Error

Initially, the POM used:

```
<version>${BUILD_NUMBER}-SNAPSHOT</version>
```

Maven rejected this because the Maven project version must be a constant value.

It was corrected to:

```
<version>1.0-SNAPSHOT</version>
```

Issue 2 — WAR Filename Mismatch

The Nexus stage initially expected a build-number-based WAR:

SimpleCustomerApp-16-SNAPSHOT.war

But Maven generated:

SimpleCustomerApp-1.0-SNAPSHOT.war

The Nexus and Tomcat stages were corrected to use:

SimpleCustomerApp-1.0-SNAPSHOT.war

After this correction, both Nexus upload and Tomcat deployment succeeded.

13. Observation

During the implementation, the Scripted Pipeline successfully automated the complete application delivery process.

The pipeline:

- Cloned the required Git branch.
- Compiled and packaged the Java web application using Maven inside Docker.
- Generated the WAR artifact.
- Performed SonarQube code analysis.
- Uploaded the WAR artifact to Nexus.
- Deployed the WAR to Tomcat.
- Verified the deployed application using HTTP response.
- Sent a completion notification to Slack.

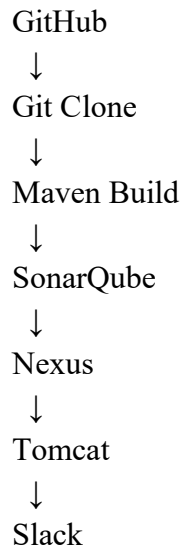
The final Jenkins build completed with:

SUCCESS

14. Conclusion

The **Jenkins Scripted Pipeline** was successfully created and executed for the SimpleCustomerApp application.

The pipeline automated the complete flow from **source code retrieval to application deployment and notification**:



The application was successfully deployed under the /hiring context and verified with an **HTTP 200 response**.

Therefore, the **Scripted Pipeline task was successfully completed** with all six required stages executing successfully.

4) Write sample skeleton of pipelines.

1. Objective

To create and execute basic **Declarative Pipeline** and **Scripted Pipeline** skeletons in Jenkins.

The purpose of this task was to understand the basic Jenkins Pipeline structure before implementing complete CI/CD pipelines.

Two pipeline types were created:

1. Declarative Pipeline
2. Scripted Pipeline

Both pipelines contained three basic stages:

Build

↓

Test

↓

Deploy

Part A – Declarative Pipeline Skeleton

2. Create Jenkins Job

1. Open the Jenkins Dashboard.
2. Click **New Item**.
3. Enter the job name:

Sample-Declarative-Pipeline

4. Select **Pipeline**.
5. Click **OK**.

3. Configure Declarative Pipeline

1. Scroll to the **Pipeline** section.
2. Select:

Pipeline script

3. Enter the following pipeline:

```
pipeline {  
  
    agent any  
  
    stages {  
  
        stage('Build') {  
            steps {  
                echo 'Build stage'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                echo 'Test stage'  
            }  
        }  
  
        stage('Deploy') {  
            steps {  
                echo 'Deploy stage'  
            }  
        }  
    }  
  
    post {  
        success {  
            echo 'Pipeline completed successfully'  
        }  
  
        failure {  
            echo 'Pipeline failed'  
        }  
    }  
}
```



```
}  
}  
}
```

4. Click **Save**.

4. Execute Declarative Pipeline

1. Open the Sample-Declarative-Pipeline job.
2. Click **Build Now**.
3. Open the generated build number.
4. Open **Console Output**.

The pipeline executes the stages in the following order:

Build

↓

Test

↓

Deploy

Expected Console Output

Build stage

Test stage

Deploy stage

Pipeline completed successfully

Finished: SUCCESS

← → ↺ ⚠ Not secure 44.199.189.60:8080/view/all/newJob

Jenkins

All / New Item

🔍 ⚙️ 👤

New Item

Enter an item name

Sample-Declarative-Pipeline

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

 **Multibranch Pipeline**

OK

Jenkins

Sample-Declarative-Pipeli... / Configure

🔍 ⚙️ 👤

Configure

 General

 Triggers

 **Pipeline**

 Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

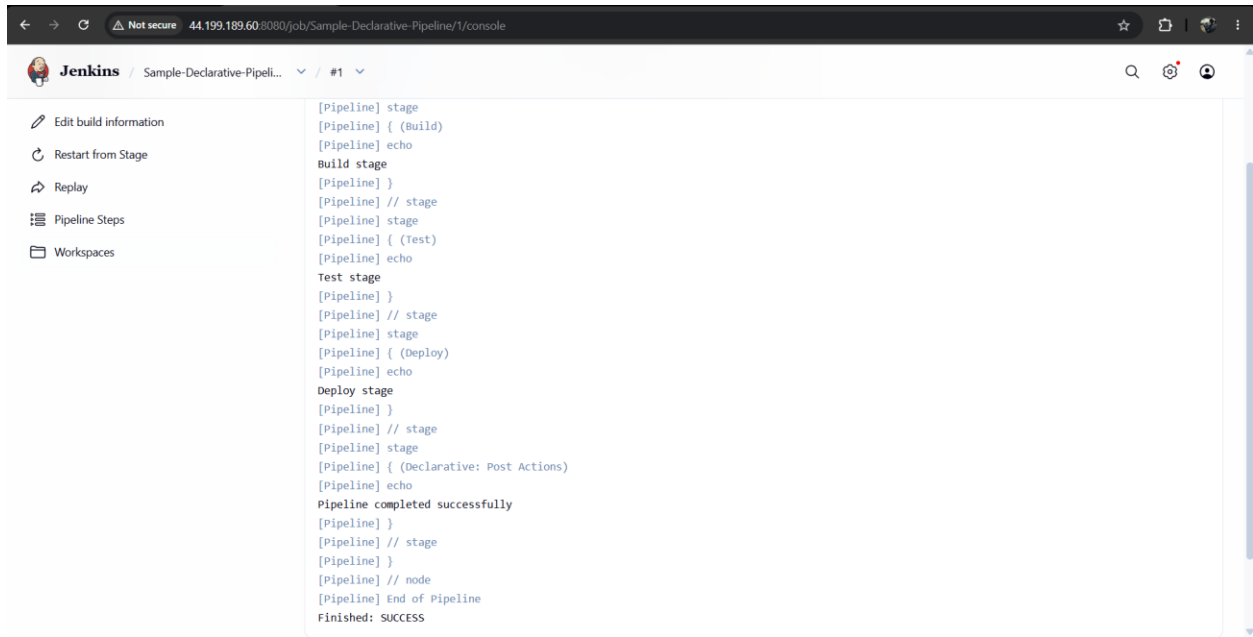
Script ?

```
1 pipeline {
2
3   agent any
4
5   stages {
6
7     stage('Build') {
8       steps {
9         echo 'Build stage'
10      }
11    }
12
13    stage('Test') {
14      steps {
15        echo 'Test stage'
16      }
17    }
18  }
19 }
```

try sample Pipeline...

Save

Apply

A screenshot of a web browser showing the Jenkins console output for a job named 'Sample-Declarative-Pipeline'. The browser's address bar shows '44.199.189.60:8080/job/Sample-Declarative-Pipeline/1/console'. The Jenkins interface includes a sidebar with options like 'Edit build information', 'Restart from Stage', 'Replay', 'Pipeline Steps', and 'Workspaces'. The main console area displays the following text:

```
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build stage
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Test stage
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Observation

The Declarative Pipeline executed all three stages successfully.

The post section executed the success block after the pipeline completed successfully.

Part B – Scripted Pipeline Skeleton

5. Create Jenkins Job

1. Return to the Jenkins Dashboard.
2. Click **New Item**.
3. Enter the job name:

Sample-Scripted-Pipeline

4. Select **Pipeline**.
5. Click **OK**.

6. Configure Scripted Pipeline

1. Scroll to the **Pipeline** section.

2. Select:

Pipeline script

3. Enter the following Scripted Pipeline:

```
node {  
  
    stage('Build') {  
        echo 'Build stage'  
    }  
  
    stage('Test') {  
        echo 'Test stage'  
    }  
  
    stage('Deploy') {  
        echo 'Deploy stage'  
    }  
  
    echo 'Pipeline completed successfully'  
}
```

4. Click **Save**.

7. Execute Scripted Pipeline

1. Open the Sample-Scripted-Pipeline job.
2. Click **Build Now**.
3. Open the generated build number.
4. Open **Console Output**.

The stages execute in this order:

Build
↓
Test
↓
Deploy

Expected Console Output

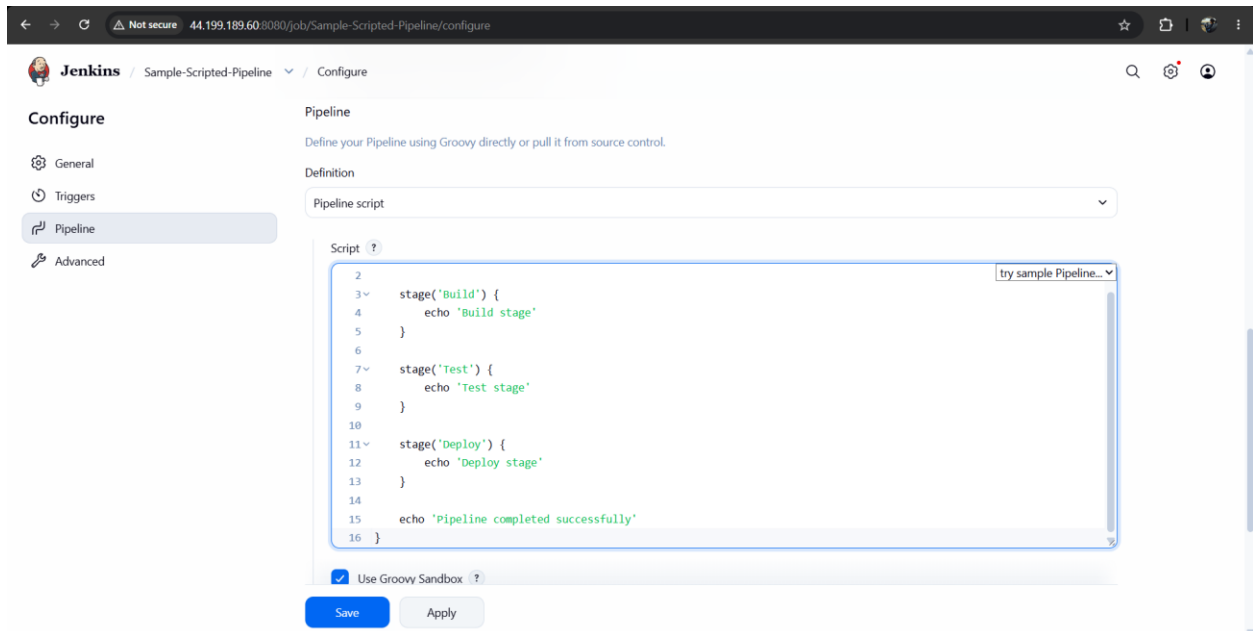
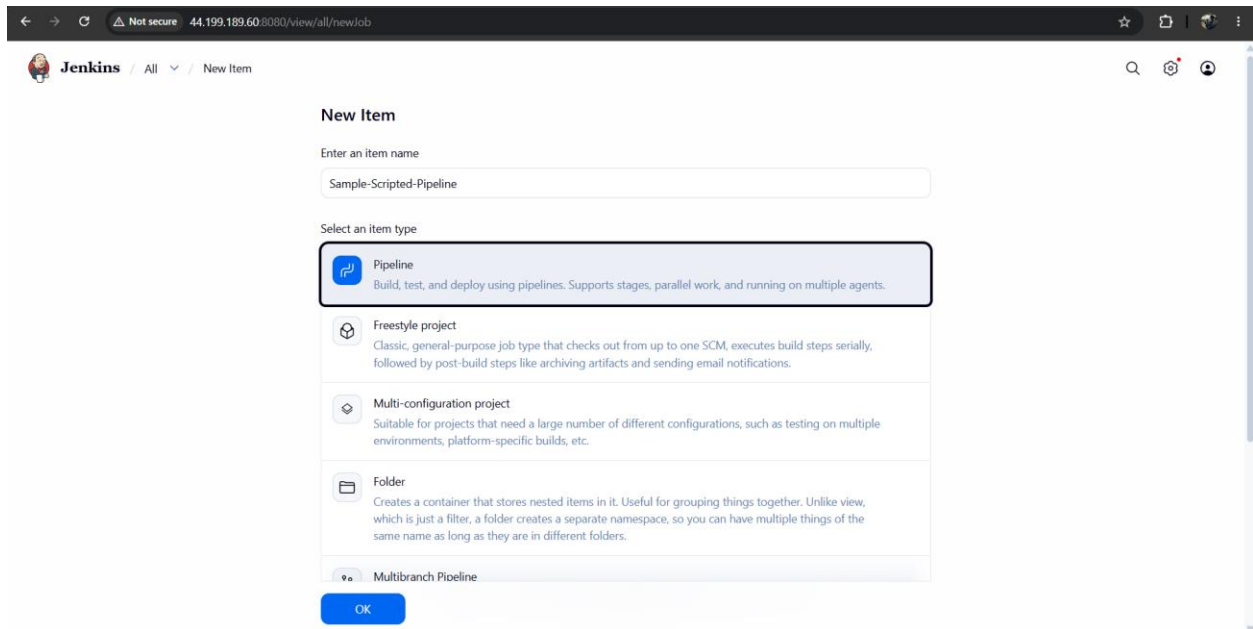
Build stage

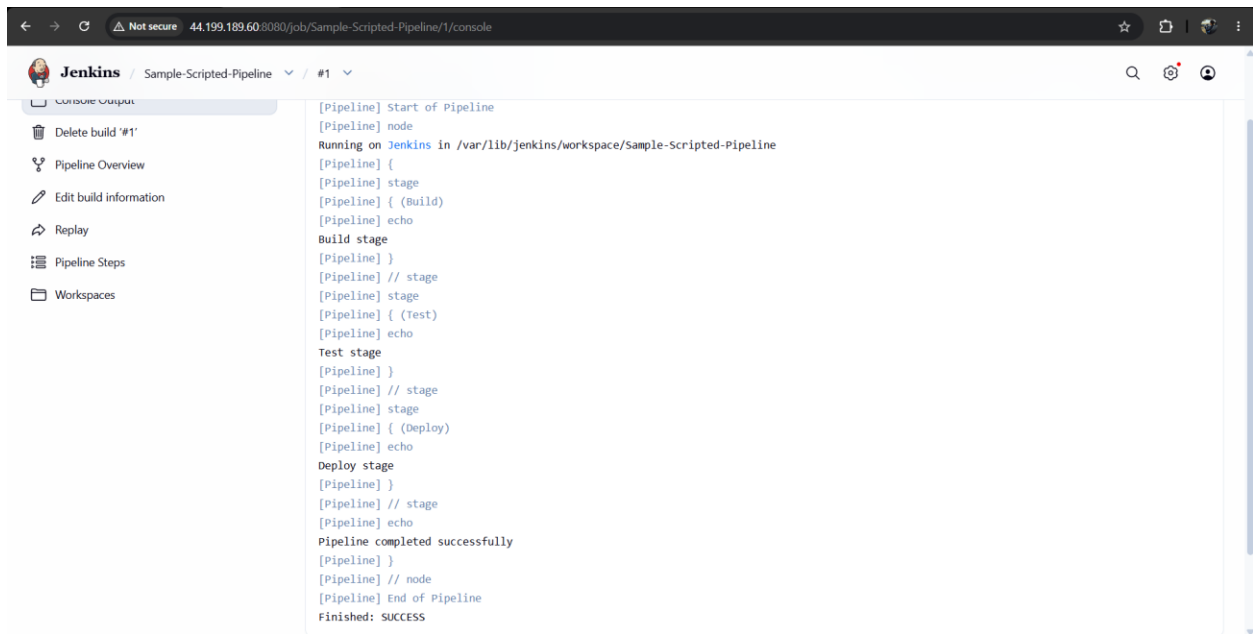
Test stage

Deploy stage

Pipeline completed successfully

Finished: SUCCESS





The screenshot shows the Jenkins web interface for a job named 'Sample-Scripted-Pipeline'. The left sidebar contains navigation links: 'Delete build #1', 'Pipeline Overview', 'Edit build information', 'Replay', 'Pipeline Steps', and 'Workspaces'. The main console area displays the following log output:

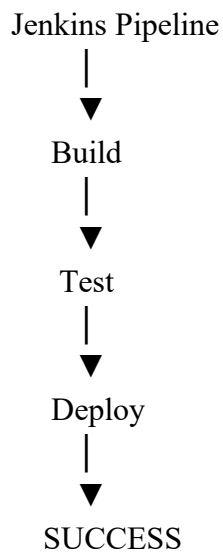
```
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Sample-Scripted-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build stage
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Test stage
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage
[Pipeline] }
[Pipeline] // stage
[Pipeline] echo
Pipeline completed successfully
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Observation

The Scripted Pipeline successfully executed all three stages using the node block.

8. Pipeline Flow

Both skeleton pipelines demonstrate the basic Jenkins Pipeline workflow:



9. Declarative vs Scripted Pipeline

Feature	Declarative	Scripted
Syntax	Structured	Groovy-based
Main structure	pipeline {}	node {}
Stages	stages {}	stage()
Ease of learning	Easier	More flexible
Flexibility	Controlled	Highly flexible
Best use	Standard CI/CD pipelines	Complex/custom logic

Simple way to remember

Declarative:

Jenkins pipeline follows a predefined structure.

Scripted:

Jenkins pipeline gives more programming flexibility using Groovy.

10. Important Jenkins Pipeline Keywords

pipeline

Defines a Declarative Pipeline.

agent any

Allows Jenkins to execute the pipeline on any available agent.

node

Allocates a Jenkins executor/workspace for a Scripted Pipeline.

stage

Defines a logical section of the pipeline.

Example:

stage('Build')

steps

Contains the commands that should execute inside a Declarative stage.

echo

Prints a message in the Jenkins console output.

post

Defines actions that should happen after the pipeline finishes.

For example:

```
post {  
    success {  
        echo 'Pipeline completed successfully'  
    }  
}
```

11. Observation

The following tasks were successfully completed:

- Created a Declarative Pipeline skeleton.
- Created a Scripted Pipeline skeleton.
- Added Build, Test, and Deploy stages.
- Executed both pipelines.
- Verified the stage execution through Console Output.
- Verified successful pipeline completion.
- Observed the basic structural difference between Declarative and Scripted Pipelines.

12. Result

Both sample Jenkins Pipeline skeletons executed successfully.

Declarative Pipeline

Build → Test → Deploy → SUCCESS

Scripted Pipeline

Build → Test → Deploy → SUCCESS

13. Conclusion

The Sample Skeleton Pipeline task provided a basic understanding of how Jenkins Pipelines are structured.

The **Declarative Pipeline** uses a predefined structure with pipeline, agent, stages, stage, steps, and post.

The **Scripted Pipeline** uses Groovy-based syntax with a node block and provides greater programming flexibility.

These basic skeletons form the foundation for creating more advanced Jenkins CI/CD pipelines containing stages such as:

Git Clone
↓
Build
↓
SonarQube
↓
Nexus
↓
Deploy
↓
Slack Notification

This task was completed successfully and prepared the foundation for the next Jenkins assignment.

5) Create a parameterized job in Jenkins.

Source Code: <https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

Objective

To create a **Parameterized Jenkins Freestyle Job** that accepts an environment value (dev, test, or prod) from the user before starting the build and then builds the application using Maven.

Source Code

GitHub Repository:

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

Branch:

master

Step 1: Create a New Jenkins Job

1. Open Jenkins.
2. Click **New Item**.
3. Enter the job name:

Parameterized-Hello-World

4. Select:

Freestyle project

5. Click **OK**.

Step 2: Configure Git Repository

Go to:

Source Code Management → Git

Configure the repository:

Repository URL:

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

Branch:

*/master

This allows Jenkins to clone the application source code from the master branch.

Step 3: Configure Build Parameter

Enable:

This project is parameterized

Click:

Add Parameter → Choice Parameter

Configure:

Name:

ENVIRONMENT

Choices:

dev

test

prod

Description:

Select the deployment environment

The parameter allows the user to select the environment before triggering the Jenkins build.

Step 4: Save the Job

Click:

Save

After saving, Jenkins displays:

Build with Parameters

instead of only **Build Now**.

This confirms that the parameterized configuration has been enabled successfully.

Step 5: Configure Execute Shell

Go to:

Configure → **Build Steps** → **Execute shell**

The following script was used:

```
echo "===== Parameterized Jenkins Job ====="
```

```
echo "Selected Environment: $ENVIRONMENT"
```

```
echo "===== Creating Compatible POM ====="
```

```
cat > pom.xml <<'EOF'<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4\_0\_0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.ncodeit</groupId>
```

```
<artifactId>ncodeit-hello-world</artifactId>
```

```
<packaging>war</packaging>
```

```
<version>3.0</version>
```

<name>spring3 mvc maven</name>

<properties>

 <maven.compiler.source>1.8</maven.compiler.source>

 <maven.compiler.target>1.8</maven.compiler.target>

 <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

</properties>

<dependencies>

 <dependency>

 <groupId>org.springframework</groupId>

 <artifactId>spring-webmvc</artifactId>

 <version>3.2.13.RELEASE</version>

 </dependency>

 <dependency>

 <groupId>javax.servlet</groupId>

 <artifactId>jstl</artifactId>

 <version>1.2</version>

 </dependency>

</dependencies>

<build>

 <plugins>

 <plugin>

 <groupId>org.apache.maven.plugins</groupId>

 <artifactId>maven-compiler-plugin</artifactId>

 <version>3.11.0</version>

 </plugin>

 <plugin>

 <groupId>org.apache.maven.plugins</groupId>

 <artifactId>maven-war-plugin</artifactId>

 <version>3.3.2</version>

 <configuration>

 <failOnMissingWebXml>>false</failOnMissingWebXml>

```
        </configuration>
      </plugin>

    </plugins>

  </build>

</project>
EOF

echo "===== Maven Build ====="

mvn clean package

echo "===== Build Completed ====="
```

Step 6: Why the Compatible POM Was Created

The original project was configured for an older Java version:

```
jdk.version = 1.6
```

The Jenkins server uses a modern Java environment, so the old Java 6 compiler configuration was not compatible.

Therefore, the Jenkins build creates a compatible pom.xml inside the Jenkins workspace with:

```
<maven.compiler.source>1.8</maven.compiler.source>
<maven.compiler.target>1.8</maven.compiler.target>
```

The POM also uses compatible Maven plugins:

```
maven-compiler-plugin 3.11.0
maven-war-plugin 3.3.2
```

This allows Maven to compile the legacy application and generate the WAR package successfully.

The original GitHub repository was not modified.

Step 7: Run the Parameterized Build

From the Jenkins job page, click:

Build with Parameters

The following options are displayed:

ENVIRONMENT

- ☐ dev
- ☐ test
- ☐ prod

Select:

dev

Click:

Build

Step 8: Verify Console Output

Open:

Build History → Build Number → Console Output

The build displays the selected environment:

```
===== Parameterized Jenkins Job =====  
Selected Environment: dev
```

Jenkins then creates the compatible POM and executes:

```
mvn clean package
```

Maven successfully compiles the source code and packages the application.

The build finishes with:

===== Build Completed =====
Finished: SUCCESS

Execution Flow

User



Build with Parameters



Select ENVIRONMENT



dev / test / prod



Git Checkout



Create Compatible POM



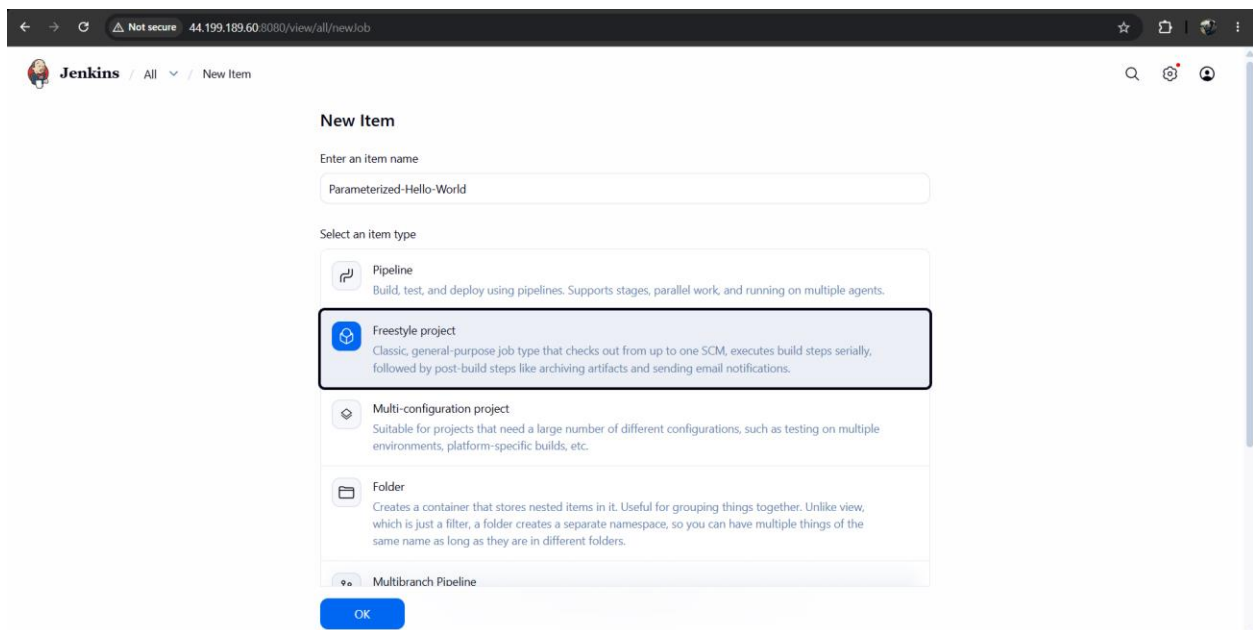
Maven Clean Package



WAR Build



Jenkins SUCCESS



← → ↻ Not secure 44.199.189.60 8080/job/Parameterized-Hello-World/configure

Jenkins / Parameterized-Hello-World / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

☐ Discard old builds

☐ GitHub project

☒ This project is parameterized

Choice Parameter

Name

ENVIRONMENT

Choices

dev
test
prod

Description

Select the deployment environment

Plain text Preview

Save Apply

← → ↻ Not secure 44.199.189.60 8080/job/Parameterized-Hello-World/configure

Jenkins / Parameterized-Hello-World / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Repositories

Repository URL

https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git

Credentials

- none - + Add

Advanced

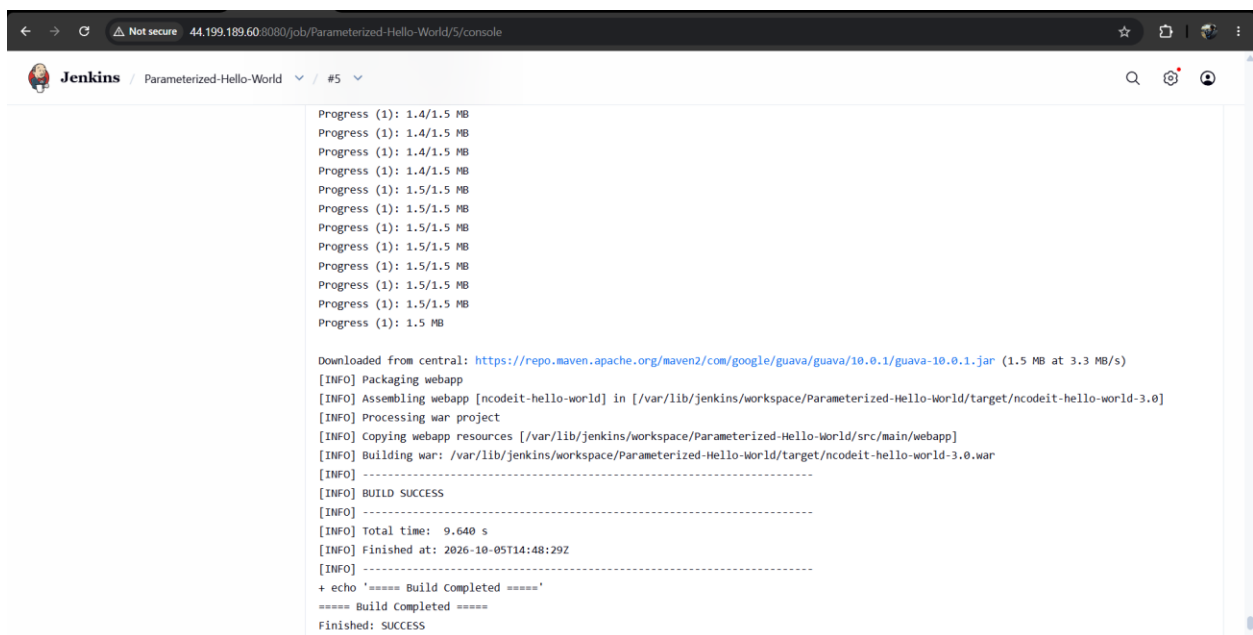
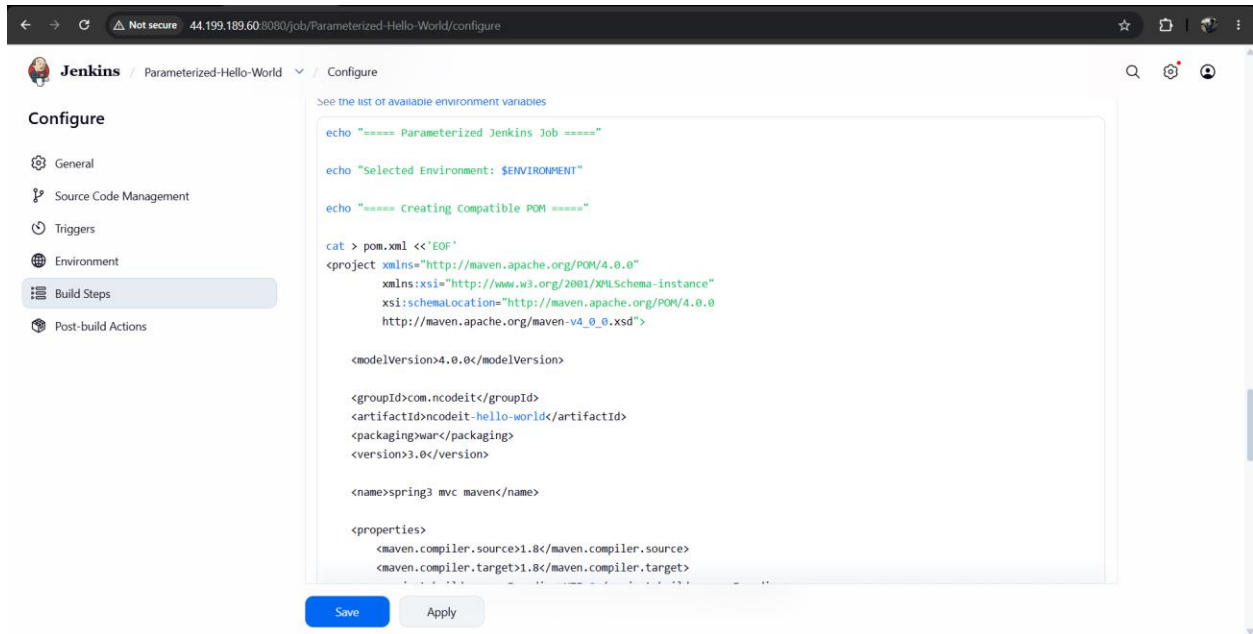
+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/master

Save Apply



Observation

1. The Jenkins job successfully displayed the **Build with Parameters** option.
2. The ENVIRONMENT parameter accepted the values:
 - a. dev
 - b. test
 - c. prod
3. The selected value was available to the shell script through:

\$ENVIRONMENT

4. The Git repository was successfully checked out from the master branch.
5. A compatible Maven POM was created in the Jenkins workspace.
6. Maven successfully compiled and packaged the application.
7. The parameterized build completed with **SUCCESS**.

Result

The **Parameterized-Hello-World** Freestyle Jenkins job was successfully created and executed.

The job accepts an environment value from the user and uses the selected parameter during the Jenkins build.

Build Result: SUCCESS 

Conclusion

A **Parameterized Jenkins Job** allows users to provide input values before triggering a build.

In this task, the ENVIRONMENT parameter was created with three choices:

dev
test
prod

When dev was selected, Jenkins received the value through the \$ENVIRONMENT environment variable and successfully executed the Maven build.

Therefore, the parameterized Jenkins Freestyle job was successfully configured and tested.

6) Setup one slave machine for Jenkins.

Jenkins Official Documentation

Recommended

- Pipeline: <https://www.jenkins.io/doc/book/pipeline/>
- Environment Variables: <https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#using-environment-variables>
- Parameters: <https://www.jenkins.io/doc/book/pipeline/syntax/#parameters>
- Handling credentials in Pipeline: <https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#handling-credentials>

Objective

To configure one Jenkins Agent and connect it to the Jenkins Controller using SSH, so that Jenkins jobs can execute remotely on the Agent machine.

1. Jenkins Architecture

The setup contains two EC2 instances:

Jenkins Controller

|
| SSH



Jenkins Agent



Execute Jenkins Jobs

Controller

- OS: Amazon Linux 2023
- Jenkins Port: 8080
- Private IP: 172.31.12.95

Agent

- Name: jenkins-agent-01
- OS: Amazon Linux 2023
- Instance Type: t3.micro
- Storage: 8 GiB gp3

- Private IP: 172.31.23.103
- Java: Corretto/OpenJDK 21
- Jenkins Agent Port: Not required

2. Launch Jenkins Agent EC2 Instance

Created an EC2 instance with:

Configuration	Value
Name	jenkins-agent-01
AMI	Amazon Linux 2023
Instance Type	t3.micro
Storage	8 GiB gp3
SSH	Port 22
SSH Source	My IP

Jenkins port 8080 was **not required** on the Agent because the Agent connects to the Controller through SSH.

3. Verify Java on Agent

Connected to the Agent using SSH and verified Java:

```
java -version
```

Output confirmed:

```
openjdk version "21.0.12.1"  
Corretto-21.0.12.9.1
```

Therefore, Java 21 was available for Jenkins Agent execution.

4. Create SSH Key Pair on Jenkins Controller

On the Jenkins Controller, created a dedicated RSA key pair:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/jenkins_agent_key
```

This created:

```
/home/ec2-user/.ssh/jenkins_agent_key  
/home/ec2-user/.ssh/jenkins_agent_key.pub
```

The private key was later configured as the Jenkins SSH credential.

5. Configure SSH Access on Agent

The Controller's public key was added to the Agent user's:

```
~/.ssh/authorized_keys
```

The SSH directory permissions were configured as:

```
chmod 700 ~/.ssh  
chmod 600 ~/.ssh/authorized_keys
```

This allowed the Jenkins Controller to authenticate to the Agent using the SSH key.

6. Verify SSH Connectivity

SSH connectivity between Controller and Agent was verified successfully using the dedicated key:

```
ssh -o IdentitiesOnly=yes \  
-i ~/.ssh/jenkins_agent_key \  
ec2-user@172.31.23.103
```

Authentication was successful.

Therefore:

Jenkins Controller

```
|  
| SSH Authentication
```

↓
172.31.23.103

was working correctly.

7. Create Jenkins Agent Node

In Jenkins:

Manage Jenkins → Nodes → New Node

Created:

Node Name: jenkins-agent-01

Selected:

Permanent Agent

Configuration:

Setting	Value
Node name	jenkins-agent-01
Executors	1
Remote root directory	/home/ec2-user/jenkins
Labels	jenkins-agent
Usage	Use this node as much as possible
Launch method	Launch agents via SSH

8. Configure SSH Credentials

Created an SSH credential:

Credential ID: jenkins-agent-01-ssh

Credential type:

SSH Username with private key

Configuration:

Username: ec2-user

Private Key: Jenkins Agent SSH private key

This credential allows Jenkins to authenticate to the Agent automatically.

9. Configure Host Key Verification

Under:

Launch agents via SSH → Host Key Verification Strategy

selected:

Manually trusted key Verification Strategy

This allowed Jenkins to trust the SSH host key of the Agent.

10. Resolve Agent /tmp Disk Monitoring Issue

Jenkins initially reported:

Disk space is below threshold of 1.00 GiB

The Agent's /tmp filesystem was only approximately:

457 MB

Verified using:

`df -h /tmp`

The /tmp filesystem was increased to 2 GB:

`sudo mount -o remount,size=2G /tmp`

Verified:

`df -h /tmp`

Final result:

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	2.0G	164K	2.0G	1%	/tmp

The mount was also verified using:

```
mount | grep ' /tmp '
```

Result confirmed:

```
size=2097152k
```

11. Restart Jenkins Agent Process

The existing Jenkins remoting process was identified using:

```
ps -ef | grep remoting.jar
```

The Jenkins Agent process was stopped so Jenkins could establish a fresh connection.

After restarting the Agent connection from Jenkins, the Agent successfully came online.

12. Verify Jenkins Agent Connection

The Jenkins launch log confirmed:

Authentication successful.

Then:

Starting sftp client.

Jenkins successfully copied:

```
remoting.jar
```

The Agent process started successfully:

Starting agent process

The important confirmation was:

Agent successfully connected and online

The Agent was therefore successfully connected to the Jenkins Controller.

13. Create Agent Test Job

Created a Freestyle Jenkins job:

Agent-Test-Job

Configured:

Restrict where this project can be run

Label:

jenkins-agent

This ensures that the job executes specifically on the Jenkins Agent.

14. Agent Test Build

Added an Execute Shell build step:

```
echo "===== Jenkins Agent Test ====="
```

```
echo "Hostname:"
```

```
hostname
```

```
echo "Current User:"
```

```
whoami
```

```
echo "Java Version:"
```

```
java -version
```

```
echo "Working Directory:"
```

```
pwd
```

```
echo "Agent Test Completed Successfully"
```

15. Test Result

The Jenkins Console Output confirmed:

Building remotely on [jenkins-agent-01]

Hostname:

ip-172-31-23-103.ec2.internal

Current user:

ec2-user

Java:

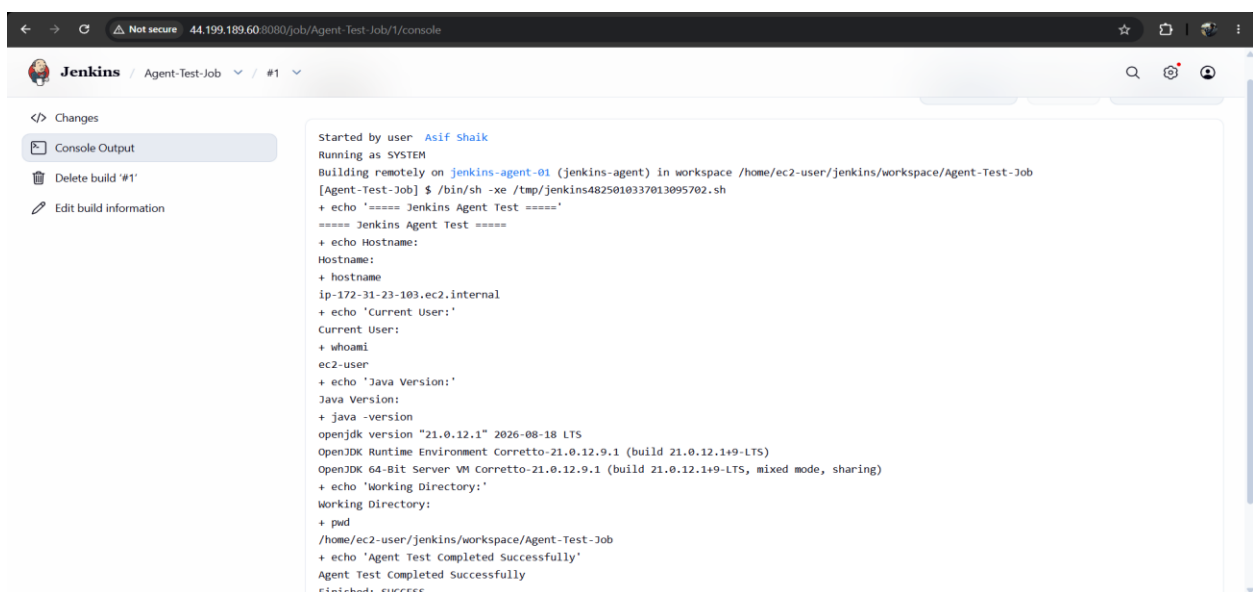
openjdk version "21.0.12.1"

Workspace:

/home/ec2-user/jenkins/workspace/Agent-Test-Job

Final build result:

Finished: SUCCESS



The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is 44.199.189.60:8080/job/Agent-Test-Job/1/console. The page title is "Jenkins / Agent-Test-Job / #1". On the left sidebar, there are links for "Changes", "Console Output", "Delete build '#1'", and "Edit build information". The main area displays the console output for build #1, which includes the following text:

```
Started by user: Asif Shaik
Running as SYSTEM
Building remotely on jenkins-agent-01 (jenkins-agent) in workspace /home/ec2-user/jenkins/workspace/Agent-Test-Job
[Agent-Test-Job] $ /bin/sh -xe /tmp/jenkins4825010337013095702.sh
+ echo '==== Jenkins Agent Test ====='
===== Jenkins Agent Test =====
+ echo Hostname:
Hostname:
ip-172-31-23-103.ec2.internal
+ echo 'Current User:'
Current User:
+ whoami
ec2-user
+ echo 'Java Version:'
Java Version:
+ java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
+ echo 'Working Directory:'
Working Directory:
+ pwd
/home/ec2-user/jenkins/workspace/Agent-Test-Job
+ echo 'Agent Test Completed Successfully'
Agent Test Completed Successfully
Finished: SUCCESS
```

Jenkins / Agent-Test-Job

Agent-Test-Job

Permalinks

- Last build (#1), 3 min 10 sec ago
- Last stable build (#1), 3 min 10 sec ago
- Last successful build (#1), 3 min 10 sec ago
- Last completed build (#1), 3 min 10 sec ago

Builds

Filter

Today

#1 4:00 PM

REST API Jenkins 2.580.1

Jenkins

Build History

Project Relationship

Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

Built-In Node 0/2

jenkins-agent-01 0/1

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	Agent-Test-Job	3 min 37 sec #1	N/A	0.95 sec
✓	☀	Hiring-App-Declarative-Pipeline	3 hr 43 min #26	4 hr 57 min #21	21 sec
✓	☁	Hiring-App-Pipeline	1 day 0 hr #12	1 day 0 hr #11	22 sec
✓	☁	Hiring-App-Scripted-Pipeline	2 hr 13 min #17	2 hr 14 min #16	23 sec
✓	☁	Parameterized-Hello-World	1 hr 16 min #5	1 hr 17 min #4	10 sec
✓	☀	Sample-Declarative-Pipeline	1 hr 48 min #1	N/A	0.66 sec
✓	☀	Sample-Scripted-Pipeline	1 hr 46 min #1	N/A	0.26 sec

Icon: S M L

Observation

The Jenkins Controller successfully connected to jenkins-agent-01 through SSH.

The test job was executed remotely on the Agent rather than on the Controller.

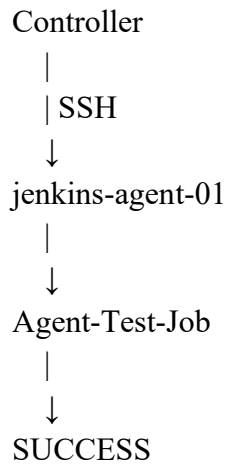
The console output confirmed the Agent hostname, user, Java version, and workspace.

The Agent successfully became **Online** and executed the Jenkins job successfully.

Result

Jenkins Agent jenkins-agent-01 was successfully configured and connected to the Jenkins Controller.

The Agent successfully executed a Jenkins Freestyle job.



Conclusion

The Jenkins Controller-Agent architecture was successfully implemented.

The Controller manages Jenkins jobs, while the Agent provides the execution environment for those jobs.

Using labels, Jenkins can direct specific jobs to the required Agent.

Therefore, the objective of **setting up one Jenkins Agent** was successfully completed.